

# BankManagement

Application Rules, Entity States, and Type Effects

---

Complete SQL-centered technical reference

This report documents every persistent entity in the integrated BankManagement schema, the declarative and procedural rules applied to it, the meaning of each type and state, the resulting authorization and user-interface behavior, and the triggers and SQL Server Agent jobs that propagate changes across related entities.

**Student:** \_\_\_\_\_

**Course:** Database Systems

**Instructor:** \_\_\_\_\_

**Date:** \_\_\_\_\_

# Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Executive Summary</b>                                    | <b>1</b>  |
| <b>1 Rule Enforcement Architecture</b>                      | <b>2</b>  |
| 1.1 Enforcement layers . . . . .                            | 2         |
| 1.2 Trusted identity and scope . . . . .                    | 2         |
| 1.3 Direct SQL access rather than ORM . . . . .             | 2         |
| 1.4 Global authorization model . . . . .                    | 3         |
| <b>2 Customer</b>                                           | <b>4</b>  |
| 2.1 Attributes and structural constraints . . . . .         | 4         |
| 2.2 Entity rules . . . . .                                  | 4         |
| 2.3 State effects . . . . .                                 | 5         |
| 2.4 Branch visibility . . . . .                             | 5         |
| 2.5 Procedures that enforce or expose these rules . . . . . | 5         |
| 2.6 Triggers and scheduled automation . . . . .             | 7         |
| <b>3 Branch</b>                                             | <b>8</b>  |
| 3.1 Attributes and structural constraints . . . . .         | 8         |
| 3.2 Entity rules . . . . .                                  | 8         |
| 3.3 Current branch and manager effects . . . . .            | 8         |
| 3.4 Balance and ledger . . . . .                            | 9         |
| 3.5 Procedures that enforce or expose these rules . . . . . | 9         |
| 3.6 Triggers and scheduled automation . . . . .             | 10        |
| <b>4 AccountType</b>                                        | <b>11</b> |
| 4.1 Attributes and structural constraints . . . . .         | 11        |
| 4.2 Entity rules . . . . .                                  | 11        |
| 4.3 Current sample product catalogue . . . . .              | 11        |
| 4.4 Procedures that enforce or expose these rules . . . . . | 12        |
| 4.5 Triggers and scheduled automation . . . . .             | 13        |
| <b>5 Account</b>                                            | <b>14</b> |
| 5.1 Attributes and structural constraints . . . . .         | 14        |
| 5.2 Entity rules . . . . .                                  | 14        |
| 5.3 AccountStatus state machine and effects . . . . .       | 15        |
| 5.4 Role/action matrix . . . . .                            | 15        |
| 5.5 Procedures that enforce or expose these rules . . . . . | 15        |
| 5.6 Triggers and scheduled automation . . . . .             | 19        |

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| <b>6</b>  | <b>Employee</b>                                         | <b>20</b> |
| 6.1       | Attributes and structural constraints . . . . .         | 20        |
| 6.2       | Entity rules . . . . .                                  | 20        |
| 6.3       | Employment state and effective-role effects . . . . .   | 21        |
| 6.4       | Hiring models . . . . .                                 | 21        |
| 6.5       | Procedures that enforce or expose these rules . . . . . | 21        |
| 6.6       | Triggers and scheduled automation . . . . .             | 25        |
| <b>7</b>  | <b>EMPB</b>                                             | <b>26</b> |
| 7.1       | Attributes and structural constraints . . . . .         | 26        |
| 7.2       | Entity rules . . . . .                                  | 26        |
| 7.3       | WorkingStatus effects . . . . .                         | 26        |
| 7.4       | Procedures that enforce or expose these rules . . . . . | 27        |
| 7.5       | Triggers and scheduled automation . . . . .             | 29        |
| <b>8</b>  | <b>TransactionType</b>                                  | <b>30</b> |
| 8.1       | Attributes and structural constraints . . . . .         | 30        |
| 8.2       | Entity rules . . . . .                                  | 30        |
| 8.3       | Type pattern . . . . .                                  | 30        |
| 8.4       | Procedures that enforce or expose these rules . . . . . | 30        |
| 8.5       | Triggers and scheduled automation . . . . .             | 32        |
| <b>9</b>  | <b>Transactions</b>                                     | <b>33</b> |
| 9.1       | Attributes and structural constraints . . . . .         | 33        |
| 9.2       | Entity rules . . . . .                                  | 33        |
| 9.3       | TransactionStatus effects . . . . .                     | 34        |
| 9.4       | Readiness tiers . . . . .                               | 34        |
| 9.5       | Procedures that enforce or expose these rules . . . . . | 34        |
| 9.6       | Triggers and scheduled automation . . . . .             | 37        |
| <b>10</b> | <b>Loan</b>                                             | <b>38</b> |
| 10.1      | Attributes and structural constraints . . . . .         | 38        |
| 10.2      | Entity rules . . . . .                                  | 38        |
| 10.3      | LoanStatus effects . . . . .                            | 38        |
| 10.4      | Visibility and payment . . . . .                        | 39        |
| 10.5      | Procedures that enforce or expose these rules . . . . . | 39        |
| 10.6      | Triggers and scheduled automation . . . . .             | 40        |
| <b>11</b> | <b>Installment</b>                                      | <b>41</b> |
| 11.1      | Attributes and structural constraints . . . . .         | 41        |
| 11.2      | Entity rules . . . . .                                  | 41        |
| 11.3      | InstallmentStatus effects . . . . .                     | 41        |
| 11.4      | Procedures that enforce or expose these rules . . . . . | 42        |
| 11.5      | Triggers and scheduled automation . . . . .             | 43        |
| <b>12</b> | <b>BranchLedger</b>                                     | <b>44</b> |
| 12.1      | Attributes and structural constraints . . . . .         | 44        |
| 12.2      | Entity rules . . . . .                                  | 44        |
| 12.3      | Trigger update algorithm . . . . .                      | 44        |
| 12.4      | Triggers and scheduled automation . . . . .             | 45        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>13 Roles</b>                                              | <b>46</b> |
| 13.1 Attributes and structural constraints . . . . .         | 46        |
| 13.2 Entity rules . . . . .                                  | 46        |
| 13.3 Assigned versus effective roles . . . . .               | 46        |
| 13.4 Procedures that enforce or expose these rules . . . . . | 47        |
| 13.5 Triggers and scheduled automation . . . . .             | 49        |
| <b>14 Users</b>                                              | <b>50</b> |
| 14.1 Attributes and structural constraints . . . . .         | 50        |
| 14.2 Entity rules . . . . .                                  | 50        |
| 14.3 Identity combinations . . . . .                         | 50        |
| 14.4 Procedures that enforce or expose these rules . . . . . | 51        |
| 14.5 Triggers and scheduled automation . . . . .             | 53        |
| <b>15 UserRoles</b>                                          | <b>54</b> |
| 15.1 Attributes and structural constraints . . . . .         | 54        |
| 15.2 Entity rules . . . . .                                  | 54        |
| 15.3 Role-combination rules . . . . .                        | 54        |
| 15.4 Procedures that enforce or expose these rules . . . . . | 54        |
| 15.5 Triggers and scheduled automation . . . . .             | 57        |
| <b>16 Sessions</b>                                           | <b>58</b> |
| 16.1 Attributes and structural constraints . . . . .         | 58        |
| 16.2 Entity rules . . . . .                                  | 58        |
| 16.3 Fixed ten-minute lifecycle . . . . .                    | 58        |
| 16.4 Procedures that enforce or expose these rules . . . . . | 59        |
| <b>17 AuditLog</b>                                           | <b>61</b> |
| 17.1 Attributes and structural constraints . . . . .         | 61        |
| 17.2 Entity rules . . . . .                                  | 61        |
| 17.3 Audit and report behavior . . . . .                     | 61        |
| 17.4 Procedures that enforce or expose these rules . . . . . | 62        |
| <b>18 EmployeeTransferRequest</b>                            | <b>80</b> |
| 18.1 Attributes and structural constraints . . . . .         | 80        |
| 18.2 Entity rules . . . . .                                  | 81        |
| 18.3 TransferStatus workflow . . . . .                       | 81        |
| 18.4 Procedures that enforce or expose these rules . . . . . | 81        |
| 18.5 Triggers and scheduled automation . . . . .             | 83        |
| <b>19 Cross-Entity Rule Chains</b>                           | <b>84</b> |
| 19.1 Signup, login, and session authorization . . . . .      | 84        |
| 19.2 Ordinary employee hiring and login creation . . . . .   | 84        |
| 19.3 HighAdmin manager hiring . . . . .                      | 84        |
| 19.4 Account opening and product effects . . . . .           | 84        |
| 19.5 Transaction posting and branch ledger . . . . .         | 84        |
| 19.6 Loan payment and default processing . . . . .           | 85        |
| 19.7 Employee transfer . . . . .                             | 85        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>20 Consolidated State and Type Matrices</b>         | <b>86</b> |
| 20.1 Closed state sets versus lookup types . . . . .   | 86        |
| 20.2 Account state operation matrix . . . . .          | 86        |
| 20.3 Employment and assignment matrix . . . . .        | 86        |
| 20.4 Loan and installment matrix . . . . .             | 87        |
| <b>21 Complete Stored Procedure Inventory</b>          | <b>88</b> |
| <b>22 Trigger, Function, View, and Agent Inventory</b> | <b>92</b> |
| 22.1 Triggers . . . . .                                | 92        |
| 22.2 Functions and helper view . . . . .               | 92        |
| 22.3 SQL Server Agent jobs . . . . .                   | 93        |
| <b>23 Conclusion</b>                                   | <b>94</b> |
| <b>Source Basis</b>                                    | <b>95</b> |

## Executive Summary

The application uses a database-centered rule model. SQL Server tables define allowed structures and closed state sets; stored procedures implement authorized state transitions; triggers protect rules that must remain valid even when a procedure is bypassed; SQL Server Agent invokes the same procedures for scheduled work; the Express backend validates the bearer session and supplies trusted identity and branch scope; and the frontend presents only actions compatible with the current role and entity state. The frontend is not the authority. Every sensitive operation is checked again by the backend and SQL Server.

The document is organized by persistent entity. Each entity chapter explains its attributes, constraints, states or types, authorization scope, procedures, automation, and impact on related entities. Lookup entities such as AccountType, TransactionType, and Roles are distinguished from lifecycle-state columns such as AccountStatus, TransactionStatus, LoanStatus, InstallmentStatus, EmpStatus, WorkingStatus, and TransferStatus.

## Chapter 1

# Rule Enforcement Architecture

### 1.1 Enforcement layers

| Layer                         | Responsibility                                                                                      | Examples                                                                                                                        |
|-------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Table constraints and indexes | Reject structurally invalid data regardless of caller.                                              | CHECK constraints for states, unique NationalID/Username/AccountNumber, one current EMPB row, one open transfer request.        |
| Stored procedures             | Perform validated, transactional state changes and write audit records.                             | Account open/freeze/close, transaction initiation/finalization/reversal, loan creation/payment, employee governance.            |
| Triggers                      | Protect cross-cutting invariants and synchronize dependent data.                                    | Transaction type pattern, pending-close protection, branch ledger, installment synchronization, role model, manager uniqueness. |
| SQL Server Agent              | Runs trusted scheduled maintenance through the same procedures used manually.                       | Pending transaction processing, overdue installments, monthly interest.                                                         |
| Express middleware/routes     | Validate bearer token, required role, request shape, and branch/ownership scope before calling SQL. | authenticate(), authorize(), access-scope.js, required body fields.                                                             |
| Frontend                      | Presents role-aware sections, state-aware action buttons, safe drop-downs, and friendly dates.      | My Accounts vs Branch Accounts, My Loans vs Branch Loans, manager-only maintenance controls.                                    |

### 1.2 Trusted identity and scope

The client never supplies the authoritative user identity. Every protected request carries an opaque bearer token. The backend calls `sp_User_ValidateSession`; the returned row becomes `req.user`. Branch and ownership helpers derive personal, branch, or all-record scope from that row. Route code passes `req.user.UserID` after request-body values so a client-supplied UserID cannot override the authenticated identity.

### 1.3 Direct SQL access rather than ORM

The backend uses the `mssql` driver, named connection pools, typed `request.input()` parameters, and `request.execute()` for named stored procedures. There are no ORM entity classes, no `schema.prisma`, no Sequelize/TypeORM model declarations, no ORM migration history, and no change-tracking layer. Therefore, the SQL objects documented here are the actual implementation of the data model and business rules.

## 1.4 Global authorization model

| Effective role | Primary scope                        | Important consequences                                                                                                                               |
|----------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer       | Own CustomerID                       | Personal accounts and loans; owner-only financial operations.                                                                                        |
| Employee       | Current BranchID plus own CustomerID | May view branch accounts, customers, loans, and appropriate employees; can manage account lifecycle in branch but cannot transact for another owner. |
| Admin          | Current BranchID plus own CustomerID | Branch Manager/Vice Manager operations, branch employee actions, overdue loan processing, unfreeze, maintenance according to routes/procedures.      |
| HighAdmin      | All branches plus own CustomerID     | Global read/governance scope; still owner-only for financial transactions and installment payments.                                                  |



## Chapter 2

# Customer

Customer is the master identity for every banking participant. Normal customers, employees, managers, and HighAdmin all ultimately link to an active Customer row so personal banking features and a consistent login identity are available.

### 2.1 Attributes and structural constraints

#### Attributes of `dbo.Customer`

| Attribute        | Type          | Constraint/default        | Application meaning                                         |
|------------------|---------------|---------------------------|-------------------------------------------------------------|
| CustomerID       | INTIDENTITY   | PK, not null              | Surrogate customer identifier.                              |
| FirstName        | NVARCHAR(50)  | Not null                  | Legal/given name.                                           |
| LastName         | NVARCHAR(50)  | Not null                  | Family name.                                                |
| NationalID       | NVARCHAR(20)  | Unique, 10 numeric digits | Stable identity key used to prevent duplicate persons.      |
| BirthDate        | DATE          | Not null                  | Used for identity and signup/hiring workflows.              |
| Phone            | NVARCHAR(20)  | Unique, not null          | Customer contact and uniqueness field.                      |
| Email            | NVARCHAR(100) | Unique, not null          | Customer contact and uniqueness field.                      |
| Address          | NVARCHAR(200) | Nullable                  | Postal/residential address.                                 |
| RegistrationDate | DATE          | Default current date      | Customer creation date.                                     |
| IsActive         | BIT           | Default 1                 | Soft-deactivation state; affects login and business access. |

### 2.2 Entity rules

- NationalID must be exactly ten numeric digits and is unique.
- Phone and Email are unique; duplicate values are rejected before insert/update.
- Deactivation is soft: IsActive becomes 0 rather than deleting historical financial records.
- A customer with any account participating in a Pending transaction cannot be deactivated; both the procedure and `TR_Customer_PreventDeactivateWithPending` protect this rule.
- Every login user must be linked to an active Customer. Staff and HighAdmin users therefore also have customer identities.
- Employee/Admin customer-directory access is limited to customers owning at least one account in the caller current branch; HighAdmin sees all.
- A newly created customer has no branch membership until an account is opened in a branch.
- Customer-only users do not receive the staff customer directory; they access their own profile through identity/session context.

## 2.3 State effects

| State      | Effect on the application                                                                                                                       |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| IsActive=1 | May own accounts and loans, may be linked to an active login, and appears in authorized directories.                                            |
| IsActive=0 | Login becomes ineffective, new operational actions are blocked, but historical accounts, loans, transactions, audit, and reporting rows remain. |

## 2.4 Branch visibility

A customer is considered a customer of a branch when at least one Account row links that CustomerID to the BranchID. A customer with accounts in multiple branches appears in each corresponding branch. This rule is applied in customer procedures and reports.

## 2.5 Procedures that enforce or expose these rules

### dbo.sp\_Customer\_Create

**Purpose.** Creates a customer master record.

**Authorization/scope.** Employee, Admin, or HighAdmin through the customer route; SQL validates the acting user.

**API use.** POST/api/customers

**Rule behavior.** Validates names, 10-digit NationalID, age/birth date, unique NationalID/-phone/email, inserts Customer, and records CustomerCreated.

**Data effects.** Reads: dbo.Customer. Writes: dbo.AuditLog, dbo.Customer.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No direct Agent job. The new customer becomes branch-visible only after an account is opened in that branch.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Create.sql

### dbo.sp\_Customer\_Search

**Purpose.** Searches customers with branch-aware visibility.

**Authorization/scope.** Employee/Admin see customers linked to their current branch through accounts; HighAdmin sees all.

**API use.** GET/api/customers

**Rule behavior.** Normalizes filters, resolves effective privilege and branch, filters by name/NationalID/phone/email/active status, and records the search.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job. Results can change after account creation, closure, or branch-related account changes.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Search.sql

r\_Search.sql

#### dbo.sp\_Customer\_GetByID

**Purpose.** Returns one authorized customer record.

**Authorization/scope.** Customer can read self; staff can read customers linked to current branch; HighAdmin can read any.

**API use.** Usedbybackend/customeroperationsanddirectSQLworkflows

**Rule behavior.** Resolves caller identity, applies self/branch/all scope, returns one customer, and records CustomerViewed.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Search.sql

#### dbo.sp\_Customer\_Update

**Purpose.** Updates customer contact information while preserving identity integrity.

**Authorization/scope.** Customer self or branch-scoped staff/HighAdmin according to SQL checks.

**API use.** PUT/api/customers/:customerID

**Rule behavior.** Locks the target, validates unique phone/email and access scope, updates allowed attributes, and records CustomerUpdated.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Users. Writes: dbo.AuditLog, dbo.Customer.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Update.sql

#### dbo.sp\_Customer\_Delete

**Purpose.** Soft-deactivates a customer and disables associated application access.

**Authorization/scope.** Scoped Employee/Admin or HighAdmin; protected by pending-financial-operation rules.

**API use.** DELETE/api/customers/:customerID

**Rule behavior.** Checks scope and business dependencies, sets Customer.IsActive=0, deactivates linked Users where appropriate, and writes an audit record.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Sessions, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog, dbo.Customer, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job. The customer trigger independently blocks deactivation when pending transactions exist.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Delete.sql

#### dbo.sp\_User\_SignUp

**Purpose.** Creates a self-service customer login and guarantees the Customer role.

**Authorization/scope.** Public endpoint; uniqueness, active-customer, and user-link rules remain enforced in SQL.

**API use.** POST/api/auth/signup

**Rule behavior.** Creates or reuses the customer identity, hashes the password with the project hash function, inserts the Users row, assigns the Customer role, and records UserSignUp.

**Data effects.** Reads: dbo.Roles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job is involved. Signup is synchronous and atomic.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_SignUp\_UPDATED.sql

## 2.6 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Customer\_PreventDeactivateWithPending, dbo.TR\_Users\_EnforceUserLinks.

## Chapter 3

# Branch

Branch defines the organizational boundary used for employee assignments, account and loan scope, branch management, reporting, and the cached branch financial balance.

### 3.1 Attributes and structural constraints

#### Attributes of `dbo.Branch`

| Attribute  | Type          | Constraint/default | Application meaning                                      |
|------------|---------------|--------------------|----------------------------------------------------------|
| BranchID   | INTIDENTITY   | PK, not null       | Branch identifier.                                       |
| BranchName | NVARCHAR(100) | Not null           | Human-readable branch name.                              |
| BranchCode | NVARCHAR(20)  | Unique, not null   | Short stable branch code.                                |
| Balance    | DECIMAL(18,2) | Default 0          | Cached aggregate maintained from account-balance deltas. |
| City       | NVARCHAR(50)  | Not null           | Branch city.                                             |
| Address    | NVARCHAR(200) | Nullable           | Branch address.                                          |
| Phone      | NVARCHAR(20)  | Nullable           | Branch phone.                                            |

### 3.2 Entity rules

- BranchCode is unique and used as a stable business key in UI/reporting.
- Branch.Balance is not manually maintained by business routes. `TR_Account_SyncBranchLedger` derives it from account balance deltas.
- Employee/Admin current branch is resolved from one EMPB row with `WorkingStatus=Working` and `EndDate IS NULL`.
- Employees and branch managers can view/act only inside their current branch; HighAdmin has all-branch scope.
- Only one active Branch Manager is allowed per branch, enforced when Employee or EMPB changes.
- Branch dropdown options expose only BranchID, BranchCode, BranchName, and City to any authenticated user; detailed branch data remains role scoped.
- Branch customer membership is indirect through `Account.CustomerID` and `Account.BranchID`, not a direct `Customer.BranchID` column.

### 3.3 Current branch and manager effects

A staff user current branch is not selected in the browser. It is returned by session validation from the current EMPB row. The branch determines which accounts, loans, customers, employees, and reports the user can access. HighAdmin has no EMPB branch and receives all-branch scope.

Only one active Branch Manager may exist in a branch. Vice Managers may coexist, but the title and `CanAccessAdmin` flag are still checked when Admin privilege is calculated.

### 3.4 Balance and ledger

TR\_Account\_SyncBranchLedger is the single point that updates Branch.Balance. Any procedure that changes Account.Balance automatically changes the branch aggregate and inserts BranchLedger evidence in the same transaction.

### 3.5 Procedures that enforce or expose these rules

#### dbo.sp\_Branch\_ListOptions

**Purpose.** Returns non-sensitive branches for UI dropdowns.

**Authorization/scope.** Any authenticated user.

**API use.** GET/api/branches/options

**Rule behavior.** Returns BranchID, code, name, and city only.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.Users.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Branch\_ListOptions.sql

#### dbo.sp\_Branch\_GetInfo

**Purpose.** Returns branch operational details according to role.

**Authorization/scope.** Employee/Admin: current branch; HighAdmin: all or selected.

**API use.** GET/api/branchesandGET/api/branches/:branchID

**Rule behavior.** Resolves current assignment, prevents cross-branch staff access, returns branch data and manager context, and audits BranchInfoViewed.

**Data effects.** Reads: dbo.Account, dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** Balances may change indirectly when transaction or interest jobs update accounts and the ledger trigger updates Branch.Balance.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Branch\_GetInfo.sql

#### dbo.sp\_Employee\_Hire

**Purpose.** Creates an ordinary employee and current branch assignment.

**Authorization/scope.** Active Branch Manager/Vice Manager for own current branch. High-Admin generic employee hiring is intentionally disabled in the route.

**API use.** POST/api/employees

**Rule behavior.** Rejects manager-level titles, derives branch from the manager, inserts Employee and EMPB Working assignment, and audits the hire. No user login is created yet.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Hire.sql

#### dbo.sp\_HighAdmin\_HireManager

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_HireManager.sql

#### dbo.sp\_HighAdmin\_ReplaceBranchManager

**Purpose.** Atomically replaces the current Branch Manager.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/branches/:branchID/replace-manager

**Rule behavior.** Locks branch/current assignment, downgrades or reassigns old manager, promotes/assigns replacement, enforces one active manager, and audits BranchManagerReplaced.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_ReplaceBranchManager.sql

## 3.6 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Account\_SyncBranchLedger, dbo.TR\_EMPB\_OneCurrentBranchManager, dbo.TR\_Employee\_OneCurrentBranchManager.

## Chapter 4

# AccountType

AccountType is an extensible lookup table. New products can be added as rows without altering the Account table or CHECK constraints. Product values influence operations rather than serving only as labels.

### 4.1 Attributes and structural constraints

#### Attributes of `dbo.AccountType`

| Attribute     | Type          | Constraint/default | Application meaning                                                          |
|---------------|---------------|--------------------|------------------------------------------------------------------------------|
| AccountTypeID | INTIDENTITY   | PK, not null       | Product identifier.                                                          |
| TypeName      | NVARCHAR(50)  | Unique, not null   | Product name shown in drop-downs and reports.                                |
| Description   | NVARCHAR(200) | Nullable           | Product description.                                                         |
| MinBalance    | DECIMAL(18,2) | Default 0          | Protected minimum used by withdrawals, transfers, opening, and type changes. |
| InterestRate  | DECIMAL(5,2)  | Default 0          | Annual percentage used by monthly interest automation.                       |
| MonthlyFee    | DECIMAL(18,2) | Default 0          | Configured product fee; stored for product policy and future fee processing. |

### 4.2 Entity rules

- TypeName is unique; products are rows rather than hard-coded account-state values.
- MinBalance controls opening eligibility, available-balance checks for outgoing operations, and type-change eligibility.
- InterestRate controls monthly interest eligibility and calculation; only Active accounts with a positive rate are processed.
- MonthlyFee is stored as product policy but the current automated fee-debit process is not implemented.
- Changing AccountType does not change AccountID or AccountNumber, but changes future minimum-balance and interest behavior.
- The safe options procedure supplies real product IDs to dropdowns; the client does not invent product identifiers.

### 4.3 Current sample product catalogue

The realistic sample data defines the following products. These are data rows, not hard-coded schema states.



| Product          | MinBalance | InterestRate | Operational impact                            |
|------------------|------------|--------------|-----------------------------------------------|
| Savings Basic    | 100        | 1.25%        | Low minimum and interest eligible.            |
| Savings Premium  | 2,500      | 2.40%        | Higher protected balance and higher interest. |
| Current Account  | 500        | 0%           | Daily account without interest.               |
| Student Account  | 50         | 0.75%        | Lowest minimum and small interest.            |
| Business Account | 5,000      | 1.10%        | High operating minimum.                       |
| Golden Reserve   | 10,000     | 3.10%        | Highest sample minimum and rate.              |

### Product change impact

Changing AccountType changes the minimum that must remain after future outgoing operations and changes future monthly interest. It does not renumber the account or rewrite historical transactions.

## 4.4 Procedures that enforce or expose these rules

### dbo.sp\_AccountType\_ListOptions

**Purpose.** Returns safe account-product rows for dropdown controls.

**Authorization/scope.** Any authenticated user.

**API use.** GET/api/accounts/options/account-types

**Rule behavior.** Returns AccountTypeID, TypeName, description, minimum balance, interest rate, and monthly fee without allowing client-defined SQL identifiers.

**Data effects.** Reads: dbo.AccountType, dbo.Customer, dbo.Users.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job; the returned InterestRate later affects the monthly-interest procedure.

**Source.** database/scripts/StoredProcedures/AccountProcedures/Procedure\_AccountType\_ListOptions.sql

### dbo.sp\_Account\_Open

**Purpose.** Opens an account for the authenticated customer.

**Authorization/scope.** Customer-owned identity; the branch and product are selected, but ownership is derived from UserID.

**API use.** POST/api/accounts

**Rule behavior.** Generates a global 12-digit zero-padded account number, validates branch/product and initial deposit, inserts the account at the product minimum balance, optionally initiates an initial deposit transaction, and audits the operation.

**Data effects.** Reads: dbo.AccountType, dbo.Branch, dbo.Customer, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.Account, dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending-transaction Agent may later finalize the optional initial deposit.

**Source.** database/scripts/StoredProcedures/AccountProcedures/Procedure\_Account\_Open.sql

#### dbo.sp\_Account\_ChangeType

**Purpose.** Changes the product attached to an account.

**Authorization/scope.** Employee/Admin within current branch or HighAdmin.

**API use.** POST/api/accounts/:accountID/change-type

**Rule behavior.** Requires Active or Dormant status, no pending transaction, a valid target AccountType, and enough available balance for the new minimum; then updates AccountTypeID and audits AccountTypeChanged.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Transactions, dbo.vw\_AccountPendingOutgoing. Writes: dbo.Account, dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** Monthly interest behavior changes on the next run because the new AccountType.InterestRate is used.

**Source.** database/scripts/StoredProcedures/AccountProcedures/Procedure\_Account\_ChangeType.sql

#### dbo.sp\_Account\_ApplyMonthlyInterest

**Purpose.** Creates monthly interest credits for eligible Active accounts.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** POST/api/accounts/maintenance/apply-monthly-interest

**Rule behavior.** Selects Active accounts whose AccountType.InterestRate is positive, calculates monthly interest, and inserts Interest transactions atomically.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called by the Agent job Apply Monthly Account Interest on day 1 at 00:05. Resulting transactions may be finalized by the pending-transaction job.

**Source.** database/scripts/StoredProcedures/AccountProcedures/Procedure\_Account\_ApplyMonthlyInterest.sql

## 4.5 Triggers and scheduled automation

**Relevant SQL Server Agent jobs:** Apply Monthly Account Interest.

## Chapter 5

### Account

Account is the central balance-bearing entity. It combines owner, branch, product, posted balance, and lifecycle state. The application separates personal ownership from operational branch visibility.

#### 5.1 Attributes and structural constraints

##### Attributes of `dbo.Account`

| Attribute            | Type          | Constraint/default                  | Application meaning                                       |
|----------------------|---------------|-------------------------------------|-----------------------------------------------------------|
| AccountID            | INTIDENTITY   | PK, not null                        | Internal account identifier.                              |
| AccountNumber        | NVARCHAR(30)  | Unique, exactly 12 numeric digits   | Customer-facing account number generated from a sequence. |
| CustomerID           | INT           | FK Customer, not null               | Owner.                                                    |
| BranchID             | INT           | FK Branch, not null                 | Operational branch.                                       |
| AccountTypeID        | INT           | FK AccountType, not null            | Current product.                                          |
| Balance              | DECIMAL(18,2) | Default 0                           | Posted balance; pending outgoing amounts are separate.    |
| OpenDate             | DATE          | Not null                            | Opening date.                                             |
| CloseDate            | DATE          | Nullable, not earlier than OpenDate | Closure date.                                             |
| AccountStatus        | NVARCHAR(20)  | Active, Closed, Frozen, Dormant     | Lifecycle state.                                          |
| FrozenPreviousStatus | NVARCHAR(20)  | NULL, Active, Dormant               | Saved state restored by unfreeze.                         |

#### 5.2 Entity rules

- AccountNumber is generated globally from `seq_AccountNumber` and formatted as a 12-digit numeric value.
- Ownership is always derived from authenticated UserID/CustomerID for account opening and financial transactions.
- Customer scope is own accounts; Employee/Admin scope is current-branch accounts; HighAdmin scope is all accounts.
- All authenticated staff roles also receive a separate personal My Accounts scope using their own CustomerID.
- Employees may freeze, change type, and close branch accounts; Admin or HighAdmin may also unfreeze. No staff privilege permits financial transactions on another owner account.
- AvailableBalance equals posted Balance minus Pending outgoing withdrawals/transfers.
- Closed accounts are excluded by default unless `IncludeClosed` is true or `Closed` is explicitly requested.
- Account cannot close while Frozen or while any Pending transaction references it; trigger-level protection covers direct SQL updates.

- Account type can change only while Active or Dormant, with no Pending transaction and enough available balance for the target MinBalance.
- Freezing preserves Active/Dormant in FrozenPreviousStatus; unfreeze restores that exact state.
- Dormant sweep considers qualifying completed customer activity and excludes bank-generated Interest activity.
- Balance changes automatically update Branch.Balance and BranchLedger in the same database transaction.

### 5.3 AccountStatus state machine and effects

| Status  | Incoming                                               | Outgoing                                                                                      | Management and automation effect                                                     |
|---------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Active  | Deposit or transfer allowed                            | Withdrawal, transfer, or installment payment allowed subject to ownership and minimum balance | May freeze, change type, close; positive-rate products may receive monthly interest. |
| Dormant | Deposit or transfer allowed and completion reactivates | Blocked                                                                                       | May freeze, change type, close; no monthly interest.                                 |
| Frozen  | Blocked                                                | Blocked                                                                                       | Only Admin or HighAdmin may unfreeze; no type change, close, or interest.            |
| Closed  | Blocked                                                | Blocked                                                                                       | Terminal; hidden unless Include-Closed; no lifecycle action or interest.             |

### 5.4 Role/action matrix

| Action                    | Owner    | Employee       | Admin          | HighAdmin |
|---------------------------|----------|----------------|----------------|-----------|
| View personal             | Yes      | Yes            | Yes            | Yes       |
| View operational          | Own only | Current branch | Current branch | All       |
| Deposit/withdraw/transfer | Own only | Own only       | Own only       | Own only  |
| Freeze                    | No       | Current branch | Current branch | All       |
| Unfreeze                  | No       | No             | Current branch | All       |
| Change type/close         | No       | Current branch | Current branch | All       |

### 5.5 Procedures that enforce or expose these rules

#### dbo.sp\_Account\_Open

**Purpose.** Opens an account for the authenticated customer.

**Authorization/scope.** Customer-owned identity; the branch and product are selected, but ownership is derived from UserID.

**API use.** POST/api/accounts

**Rule behavior.** Generates a global 12-digit zero-padded account number, validates branch/product and initial deposit, inserts the account at the product minimum balance, optionally initiates an initial deposit transaction, and audits the operation.

**Data effects.** Reads: `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending-transaction Agent may later finalize the optional initial deposit.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Open.sql`

#### `dbo.sp_Account_Search`

**Purpose.** Searches account portfolios with self, branch, or all-account scope.

**Authorization/scope.** Customer: own; Employee/Admin: current branch; HighAdmin: all.

**API use.** `GET/api/accountsandGET/api/accounts/mineviaaccess-scope.js`

**Rule behavior.** Applies rich filters, blocks customer cross-account access, calculates pending outgoing amount and available balance, orders results, and records `AccountSearch`.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No direct Agent call. Pending-batch completion can change posted and available balances between searches.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Search.sql`

#### `dbo.sp_Account_GetInfo`

**Purpose.** Returns one account with ownership/branch/all authorization.

**Authorization/scope.** Same scope model as account search.

**API use.** `GET/api/accounts/:accountID`

**Rule behavior.** Resolves the caller and target, rejects unauthorized direct IDs, returns account/customer/branch/type details and available balance, and records `ACCOUNT_VIEW`.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Employee`, `dbo.Roles`, `dbo.TransactionType`, `dbo.Transactions`, `dbo.UserRoles`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_GetInfo.sql`

#### `dbo.sp_Transaction_GetAccountHistory`

**Purpose.** Returns newest-first transaction history for one authorized account.

**Authorization/scope.** Self, current branch, or all depending on effective role and requested scope.

**API use.** `GET/api/accounts/:accountID/history`

**Rule behavior.** Validates date range and account visibility, then returns debit and credit participation with transaction timestamps/status.

**Data effects.** Reads: `dbo.Account`, `dbo.Users`, `dbo.vw_AccountTransactionHistory`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The pending transaction Agent changes Pending rows to Completed/Failed, which is reflected in subsequent history reads.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountHistory.sql`

#### `dbo.sp_Account_GetAvailableBalance`

**Purpose.** Calculates spendable balance after pending outgoing holds.

**Authorization/scope.** Authorized account viewer.

**API use.** Used by account and transaction workflows

**Rule behavior.** Subtracts pending outgoing withdrawals/transfers from posted balance. This prevents double spending while transactions wait for finalization.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The pending transaction Agent releases the hold by completing or failing the transaction.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountHistory.sql`

#### `dbo.sp_Account_Freeze`

**Purpose.** Freezes an account while remembering whether it was Active or Dormant.

**Authorization/scope.** Employee/Admin within current branch; HighAdmin across all branches.

**API use.** `POST/api/accounts/:accountID/freeze`

**Rule behavior.** Locks the account, rejects Closed/already Frozen states, copies `AccountStatus` to `FrozenPreviousStatus`, sets `Frozen`, and audits `AccountFrozen`.

**Data effects.** Reads: `dbo.Account`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job directly changes Frozen accounts.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql`

#### `dbo.sp_Account_Unfreeze`

**Purpose.** Restores a frozen account to its previous Active or Dormant status.

**Authorization/scope.** Admin/branch manager within current branch or HighAdmin.

**API use.** `POST/api/accounts/:accountID/unfreeze`

**Rule behavior.** Locks the account, requires `Frozen` status, restores `FrozenPreviousStatus`,

clears the saved value, and audits AccountUnfrozen.

**Data effects.** Reads: `dbo.Account`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql`

#### `dbo.sp_Account_Close`

**Purpose.** Closes an account safely.

**Authorization/scope.** Employee/Admin within current branch or HighAdmin; no financial transaction privilege is implied.

**API use.** `POST/api/accounts/:accountID/close`

**Rule behavior.** Locks the account, rejects Frozen/already Closed/pending-transaction states, sets balance to zero and status Closed with CloseDate, and audits AccountClosed.

**Data effects.** Reads: `dbo.Account`, `dbo.Transactions`, `dbo.Users`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** The account-ledger trigger records the balance delta automatically. The pending Agent must clear pending transactions before closure is allowed.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Close.sql`

#### `dbo.sp_Account_ChangeType`

**Purpose.** Changes the product attached to an account.

**Authorization/scope.** Employee/Admin within current branch or HighAdmin.

**API use.** `POST/api/accounts/:accountID/change-type`

**Rule behavior.** Requires Active or Dormant status, no pending transaction, a valid target AccountType, and enough available balance for the new minimum; then updates AccountTypeID and audits AccountTypeChanged.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Transactions`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** Monthly interest behavior changes on the next run because the new AccountType.InterestRate is used.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ChangeType.sql`

#### `dbo.sp_Account_DormantSweep`

**Purpose.** Marks inactive eligible accounts Dormant.

**Authorization/scope.** Manual Admin/HighAdmin maintenance endpoint.

**API use.** `POST/api/accounts/maintenance/dormant-sweep`

**Rule behavior.** Finds Active accounts without qualifying completed customer activity for the configured month threshold, updates status, and audits each transition. Interest transactions do not count as customer activity.

**Data effects.** Reads: `dbo.Account`, `dbo.TransactionType`, `dbo.Transactions`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** There is no SQL Server Agent script for the dormant sweep in the current project; it is manual maintenance.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_DormantSweep.sql`

#### `dbo.sp_Account_ApplyMonthlyInterest`

**Purpose.** Creates monthly interest credits for eligible Active accounts.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** `POST/api/accounts/maintenance/apply-monthly-interest`

**Rule behavior.** Selects Active accounts whose `AccountType.InterestRate` is positive, calculates monthly interest, and inserts Interest transactions atomically.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.TransactionType`. Writes: `dbo.AuditLog`, `dbo.Transactions`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called by the Agent job Apply Monthly Account Interest on day 1 at 00:05. Resulting transactions may be finalized by the pending-transaction job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ApplyMonthlyInterest.sql`

## 5.6 Triggers and scheduled automation

**Relevant triggers:** `dbo.TR_Account_PreventCloseWithPending`, `dbo.TR_Account_SyncBranchLedger`.

**Relevant SQL Server Agent jobs:** Apply Monthly Account Interest, Process Pending Bank Transactions.



## Chapter 6

# Employee

Employee stores employment facts. Its status, title, admin flag, and current EMPB assignment determine whether Employee or Admin roles are effective.

### 6.1 Attributes and structural constraints

#### Attributes of `dbo.Employee`

| Attribute      | Type          | Constraint/default            | Application meaning                                               |
|----------------|---------------|-------------------------------|-------------------------------------------------------------------|
| EmployeeID     | INTIDENTITY   | PK, not null                  | Employee identifier.                                              |
| NationalID     | NVARCHAR(20)  | Unique, not null              | Employee identity.                                                |
| FirstName      | NVARCHAR(50)  | Not null                      | Given name.                                                       |
| LastName       | NVARCHAR(50)  | Not null                      | Family name.                                                      |
| HireDate       | DATE          | Not null                      | Employment start.                                                 |
| JobTitle       | NVARCHAR(100) | Not null                      | Operational title; manager titles affect Admin role.              |
| Salary         | DECIMAL(18,2) | Not null                      | Salary.                                                           |
| Phone          | NVARCHAR(20)  | Nullable                      | Contact.                                                          |
| Email          | NVARCHAR(100) | Nullable                      | Contact.                                                          |
| EmpStatus      | NVARCHAR(20)  | Active, OnLeave, Terminated   | Employment state.                                                 |
| CanAccessAdmin | BIT           | Default 0; trigger-maintained | Manager-level capability flag synchronized from title and status. |

### 6.2 Entity rules

- EmpStatus is limited to Active, OnLeave, or Terminated.
- CanAccessAdmin is trigger-maintained: it is 1 only for Active Branch Manager/Vice Manager employees.
- Ordinary employee hiring is performed by an active manager in the manager current branch; manager-level titles are rejected.
- HighAdmin manager hiring requires an explicit valid BranchID and creates customer, employee, assignment, user, and roles atomically.
- Ordinary employee hiring creates Employee and EMPB only; login creation is a separate controlled operation.
- Employee/Admin effective roles disappear when EmpStatus is OnLeave or Terminated, but Customer access can remain.
- Normal Employee may view self; Admin views current-branch employees; Vice Manager cannot govern the Branch Manager; HighAdmin sees all.

- Ordinary management procedures cannot promote someone to Branch Manager/Vice Manager; those titles are reserved for HighAdmin governance.
- Firing sets EmpStatus=Terminated and closes the current EMPB assignment as Ended.
- Suspension sets OnLeave but keeps the current EMPB assignment, preserving organizational placement.

## 6.3 Employment state and effective-role effects

| EmpStatus  | EMPB effect                         | Application effect                                                                                |
|------------|-------------------------------------|---------------------------------------------------------------------------------------------------|
| Active     | Current Working row normally exists | Employee role effective; Admin effective only for manager title, flag, roles, and current branch. |
| OnLeave    | Working row remains open            | Employee/Admin roles ineffective; Customer role may remain.                                       |
| Terminated | Current row becomes Ended           | Employee/Admin ineffective; no current branch; historical records retained.                       |

## 6.4 Hiring models

An Admin hires an ordinary employee into the manager current branch; the employee login is created later. HighAdmin hires Branch Manager/Vice Manager through the dedicated manager procedure with mandatory BranchID; the operation creates or reuses Customer, inserts Employee and EMPB, creates/reuses Users, and assigns Customer, Employee, and Admin in one transaction.

## 6.5 Procedures that enforce or expose these rules

### dbo.sp\_Employee\_Hire

**Purpose.** Creates an ordinary employee and current branch assignment.

**Authorization/scope.** Active Branch Manager/Vice Manager for own current branch. High-Admin generic employee hiring is intentionally disabled in the route.

**API use.** POST/api/employees

**Rule behavior.** Rejects manager-level titles, derives branch from the manager, inserts Employee and EMPB Working assignment, and audits the hire. No user login is created yet.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Hire.sql

### dbo.sp\_Employee\_CreateUserAccount

**Purpose.** Creates an application login for an existing ordinary employee.

**Authorization/scope.** Scoped Admin or HighAdmin, with branch and manager-level restrictions.

**API use.** POST/api/employees/:employeeID/create-user-account

**Rule behavior.** Creates/reuses Customer, creates/reuses Users, links EmployeeID/CustomerID,

assigns Customer and Employee roles, and audits EmployeeUserAccountCreated.

**Data effects.** Reads: `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.UserRoles`, `dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_Employee_CreateUserAccount.sql`

### `dbo.sp_Employee_Search`

**Purpose.** Searches employee records with strict target scope.

**Authorization/scope.** Normal Employee: self; Admin: current branch; Vice Manager cannot inspect Branch Manager; HighAdmin: all.

**API use.** `GET/api/employees`

**Rule behavior.** Normalizes filters, optionally searches assignment history, applies current-branch rules, and audits the search.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_Employee_Search.sql`

### `dbo.sp_Employee_GetInfo`

**Purpose.** Returns one employee and current assignment details.

**Authorization/scope.** Self/current branch/all depending on effective role, including Vice Manager protection.

**API use.** `GET/api/employees/:employeeID`

**Rule behavior.** Resolves target scope and returns employee information, current branch, and administrative flags; audits EmployeeInfoViewed.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_Employee_GetInfo.sql`

### `dbo.sp_Employee_ChangeJobTitle`

**Purpose.** Changes an ordinary employee job title.

**Authorization/scope.** Scoped Admin/HighAdmin; manager-level titles are reserved for High-Admin governance procedures.

**API use.** `POST/api/employees/:employeeID/change-job-title`

**Rule behavior.** Locks target employee, validates branch and title restrictions, updates JobTitle, lets the admin-flag trigger synchronize CanAccessAdmin, and audits.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_ChangeJobTitle.sql

#### dbo.sp\_Employee\_Suspend

**Purpose.** Places an employee OnLeave.

**Authorization/scope.** Scoped Admin/HighAdmin; managers cannot suspend themselves and ordinary procedure cannot govern manager hierarchy improperly.

**API use.** POST/api/employees/:employeeID/suspend

**Rule behavior.** Locks target, validates current status/scope, sets EmpStatus=OnLeave, and audits. Session validation then removes effective Employee/Admin privileges while preserving Customer access.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job; role loss is enforced on the next request.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Suspend.sql

#### dbo.sp\_Employee\_Fire

**Purpose.** Terminates an employee and closes the current assignment.

**Authorization/scope.** Scoped Admin/HighAdmin with self/manager restrictions.

**API use.** POST/api/employees/:employeeID/fire

**Rule behavior.** Sets EmpStatus=Terminated, ends current EMPB row, and audits. The customer login may remain active but no longer receives effective staff roles.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Fire.sql

#### dbo.sp\_HighAdmin\_HireManager

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.Employee`, `dbo.UserRoles`, `dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_HireManager.sql`

#### `dbo.sp_HighAdmin_PromoteToManager`

**Purpose.** Promotes an active employee with login to manager level.

**Authorization/scope.** HighAdmin only.

**API use.** `POST/api/highadmin/managers/:employeeID/promote`

**Rule behavior.** Validates target and branch, updates title/admin capability, adjusts assignment when necessary, ensures Admin role, and audits `EmployeePromotedToManager`.

**Data effects.** Reads: `dbo.Branch`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.Employee`, `dbo.UserRoles`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_PromoteToManager.sql`

#### `dbo.sp_HighAdmin_DowngradeManager`

**Purpose.** Removes manager status and returns the employee to an ordinary title.

**Authorization/scope.** HighAdmin only.

**API use.** `POST/api/highadmin/managers/:employeeID/downgrade`

**Rule behavior.** Validates a non-manager replacement title, updates `JobTitle/CanAccessAdmin`, removes effective Admin behavior through triggers/role rules, and audits `ManagerDowngraded`.

**Data effects.** Reads: `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.Employee`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_DowngradeManager.sql`

#### `dbo.sp_HighAdmin_SuspendManager`

**Purpose.** Places a manager OnLeave.

**Authorization/scope.** HighAdmin only.

**API use.** `POST/api/highadmin/managers/:employeeID/suspend`

**Rule behavior.** Validates manager target, updates status, and audits ManagerSuspended. Effective Admin/Employee roles disappear on session revalidation.

**Data effects.** Reads: `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.Employee`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_SuspendManager.sql`

#### `dbo.sp_HighAdmin_FireManager`

**Purpose.** Terminates a manager and closes branch assignment.

**Authorization/scope.** HighAdmin only.

**API use.** `POST/api/highadmin/managers/:employeeID/fire`

**Rule behavior.** Sets Terminated, ends current EMPB, and audits ManagerFired while preserving customer identity.

**Data effects.** Reads: `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.Employee`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_FireManager.sql`

#### `dbo.sp_HighAdmin_ReplaceBranchManager`

**Purpose.** Atomically replaces the current Branch Manager.

**Authorization/scope.** HighAdmin only.

**API use.** `POST/api/highadmin/branches/:branchID/replace-manager`

**Rule behavior.** Locks branch/current assignment, downgrades or reassigns old manager, promotes/assigns replacement, enforces one active manager, and audits BranchManagerReplaced.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.Employee`, `dbo.UserRoles`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_ReplaceBranchManager.sql`

## 6.6 Triggers and scheduled automation

**Relevant triggers:** `dbo.TR_Employee_SetAdminFlag`, `dbo.TR_Employee_OneCurrentBranchManager`.

## Chapter 7

# EMPB

EMPB is a temporal bridge between Employee and Branch. Historical periods are preserved rather than overwriting the branch on the Employee row.

### 7.1 Attributes and structural constraints

#### Attributes of `dbo.EMPB`

| Attribute     | Type         | Constraint/default                      | Application meaning    |
|---------------|--------------|-----------------------------------------|------------------------|
| EMPBID        | INTIDENTITY  | PK, not null                            | Assignment identifier. |
| EmployeeID    | INT          | FK Employee, not null                   | Assigned employee.     |
| BranchID      | INT          | FK Branch, not null                     | Assigned branch.       |
| StartDate     | DATE         | Not null                                | Assignment start.      |
| EndDate       | DATE         | Nullable, not earlier than<br>StartDate | Assignment end.        |
| WorkingStatus | NVARCHAR(20) | Working/Transferred/Ended               | Assignment state.      |

### 7.2 Entity rules

- Working means current assignment; Transferred means the period ended because a new branch assignment followed; Ended means the period ended without transfer.
- EndDate cannot precede StartDate.
- A filtered unique index permits only one current Working/EndDate NULL assignment per employee.
- The overlap trigger blocks intersecting assignment date ranges for the same employee.
- CurrentBranchID used by authorization comes only from the current Working row.
- Approved destination transfer closes the old row as Transferred and inserts a new Working row.
- Termination closes the current row as Ended without creating a replacement.
- Only one active Branch Manager may exist in a branch across current EMPB rows.

### 7.3 WorkingStatus effects

| Status      | Meaning                                          | Application consequence                                                              |
|-------------|--------------------------------------------------|--------------------------------------------------------------------------------------|
| Working     | Current assignment, EndDate NULL                 | Supplies CurrentBranchID and current manager/employee membership.                    |
| Transferred | Historical assignment replaced by another branch | Old branch loses operational scope; new Working row supplies the destination branch. |
| Ended       | Historical assignment ended without replacement  | No current branch; normally follows termination.                                     |

## 7.4 Procedures that enforce or expose these rules

### dbo.sp\_EMPBranchHistory\_Get

**Purpose.** Returns temporal branch assignment history.

**Authorization/scope.** Self/current branch/all with manager hierarchy protection.

**API use.** GET/api/employees/:employeeID/branch-history

**Rule behavior.** Reads EMPB rows, optionally current-only, validates branch scope and target hierarchy, and audits EMPBHistoryViewed.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EMPBranchHistory\_Get.sql

### dbo.sp\_Employee\_Hire

**Purpose.** Creates an ordinary employee and current branch assignment.

**Authorization/scope.** Active Branch Manager/Vice Manager for own current branch. High-Admin generic employee hiring is intentionally disabled in the route.

**API use.** POST/api/employees

**Rule behavior.** Rejects manager-level titles, derives branch from the manager, inserts Employee and EMPB Working assignment, and audits the hire. No user login is created yet.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Hire.sql

### dbo.sp\_Employee\_Fire

**Purpose.** Terminates an employee and closes the current assignment.

**Authorization/scope.** Scoped Admin/HighAdmin with self/manager restrictions.

**API use.** POST/api/employees/:employeeID/fire

**Rule behavior.** Sets EmpStatus=Terminated, ends current EMPB row, and audits. The customer login may remain active but no longer receives effective staff roles.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Fire.sql



**dbo.sp\_EmployeeTransfer\_ApproveDestinationManager**

**Purpose.** Performs destination approval and executes the transfer.

**Authorization/scope.** Destination Branch Manager/Vice Manager.

**API use.** POST/api/employee-transfers/:id/destination-manager-decision

**Rule behavior.** Locks request and assignment, records decision, and on approval ends old EMPB, creates new Working EMPB, sets EffectiveDate/CompletedAt, and audits.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job. EMPB overlap and one-manager triggers enforce temporal integrity.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_ApproveDestinationManager.sql

**dbo.sp\_HighAdmin\_HireManager**

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_HireManager.sql

**dbo.sp\_HighAdmin\_PromoteToManager**

**Purpose.** Promotes an active employee with login to manager level.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/promote

**Rule behavior.** Validates target and branch, updates title/admin capability, adjusts assignment when necessary, ensures Admin role, and audits EmployeePromotedToManager.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_PromoteToManager.sql

#### dbo.sp\_HighAdmin\_ReplaceBranchManager

**Purpose.** Atomically replaces the current Branch Manager.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/branches/:branchID/replace-manager

**Rule behavior.** Locks branch/current assignment, downgrades or reassigns old manager, promotes/assigns replacement, enforces one active manager, and audits BranchManagerReplaced.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_ReplaceBranchManager.sql

## 7.5 Triggers and scheduled automation

**Relevant triggers:** dbo.trg\_EMPB\_PreventDateOverlap, dbo.TR\_EMPB\_OneCurrentBranchManager.

## Chapter 8

# TransactionType

TransactionType is an extensible category lookup whose current seeded rows are Deposit, Withdrawal, Transfer, and Interest. A trigger converts each category into a structural debit/credit rule.

### 8.1 Attributes and structural constraints

#### Attributes of dbo.TransactionType

| Attribute         | Type         | Constraint/default | Application meaning                         |
|-------------------|--------------|--------------------|---------------------------------------------|
| TransactionTypeID | INTIDENTITY  | PK, not null       | Transaction category identifier.            |
| TypeName          | NVARCHAR(50) | Unique, not null   | Deposit, Withdrawal, Transfer, or Interest. |

### 8.2 Entity rules

- Deposit and Interest require ToAccountID and prohibit FromAccountID.
- Withdrawal requires FromAccountID and prohibits ToAccountID.
- Transfer requires both accounts and the accounts must differ.
- TR\_Transactions\_EnforceTypePattern applies the shape rule on insert and update, protecting direct SQL paths.
- Interest is bank-generated and cannot be initiated through the customer transaction UI.

### 8.3 Type pattern

| Type       | FromAccountID | ToAccountID         | Completed effect                                |
|------------|---------------|---------------------|-------------------------------------------------|
| Deposit    | NULL          | Required            | Credit destination.                             |
| Withdrawal | Required      | NULL                | Debit source.                                   |
| Transfer   | Required      | Required, different | Debit source and credit destination atomically. |
| Interest   | NULL          | Required            | Bank-generated credit.                          |

### 8.4 Procedures that enforce or expose these rules

#### dbo.sp\_Transaction\_Deposit

**Purpose.** Initiates an owner-authorized deposit.

**Authorization/scope.** Only the account owner; staff roles do not grant transaction authority.

**API use.** POST/api/transactions/deposit

**Rule behavior.** Validates caller/customer ownership, positive amount, destination state, transaction type, computes ReadyToCompleteAt, inserts Pending transaction, and audits DepositInitiated.

**Data effects.** Reads: dbo.Account, dbo.TransactionType, dbo.Users. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The Process Pending Bank Transactions Agent finalizes it when ready.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Desposit\_SP.sql

#### dbo.sp\_Transaction\_Withdrawal

**Purpose.** Initiates an owner-authorized withdrawal.

**Authorization/scope.** Only the account owner.

**API use.** POST/api/transactions/withdraw

**Rule behavior.** Locks account and pending holds, requires Active source, positive amount, and enough available balance to preserve AccountType.MinBalance; inserts Pending withdrawal and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent finalizes it; until then the amount is reserved in available balance.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Withdraw\_SP.sql

#### dbo.sp\_Transaction\_Transfer

**Purpose.** Initiates an owner-authorized transfer.

**Authorization/scope.** Only the source-account owner; destination ownership is not required.

**API use.** POST/api/transactions/transfer

**Rule behavior.** Validates distinct accounts, source Active, destination not Closed/Frozen, source ownership, available funds/minimum balance, computes delay, inserts Pending transfer, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent posts both account balance changes when ReadyToCompleteAt is reached.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Transfer\_SP.sql

#### dbo.sp\_Account\_ApplyMonthlyInterest

**Purpose.** Creates monthly interest credits for eligible Active accounts.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** POST/api/accounts/maintenance/apply-monthly-interest

**Rule behavior.** Selects Active accounts whose AccountType.InterestRate is positive, calculates monthly interest, and inserts Interest transactions atomically.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called by the Agent job Apply Monthly Account Interest on day 1 at 00:05. Resulting transactions may be finalized by the pending-transaction job.

**Source.** database/scripts/StoredProcedure/AccountProcedures/Procedure\_Account\_ApplyMonthlyInterest.sql

## 8.5 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Transactions\_EnforceTypePattern.

## Chapter 9

# Transactions

Transactions is the monetary workflow table. It records requests before posting, readiness time, completion/failure/cancellation, and the debit/credit accounts involved.

### 9.1 Attributes and structural constraints

#### Attributes of `dbo.Transactions`

| Attribute         | Type          | Constraint/default                    | Application meaning                                 |
|-------------------|---------------|---------------------------------------|-----------------------------------------------------|
| TransactionID     | INTIDENTITY   | PK, not null                          | Transaction identifier.                             |
| TransactionTypeID | INT           | FK TransactionType                    | Category controlling account-reference shape.       |
| FromAccountID     | INT           | Nullable FK Account                   | Debit/source side.                                  |
| ToAccountID       | INT           | Nullable FK Account                   | Credit/destination side.                            |
| EmployeeID        | INT           | Nullable FK Employee                  | Optional staff identity associated with initiation. |
| Amount            | DECIMAL(18,2) | Positive                              | Monetary amount.                                    |
| TransactionDate   | DATETIME      | Default current time                  | Creation timestamp.                                 |
| TransactionStatus | NVARCHAR(20)  | Pending, Completed, Failed, Cancelled | Processing state.                                   |
| ReadyToCompleteAt | DATETIME      | Nullable                              | Earliest normal finalization time.                  |
| CompletedAt       | DATETIME      | Required only when Completed          | Posting timestamp.                                  |
| Description       | NVARCHAR(200) | Nullable                              | Narrative.                                          |

### 9.2 Entity rules

- Amount must be positive and at least one account reference must be present.
- Only the owner can initiate Deposit, Withdrawal, Transfer, manual finalization, or reversal for the owner portfolio.
- Deposit destination may be Active or Dormant; Withdrawal source must be Active; Transfer source must be Active and destination Active or Dormant.
- Frozen and Closed accounts reject all normal financial participation.
- Pending outgoing amounts are holds and reduce available balance immediately before posted Balance changes.
- ReadyToCompleteAt is amount based: <5000=0s, <25000=60s, <100000=180s, otherwise 300s.
- Completed requires CompletedAt; non-Completed rows must have CompletedAt NULL.
- Finalization rechecks current account status, ownership when user-driven, minimum balance, and destination validity; it does not trust initiation-time state.

- ProcessPendingBatch is Admin or HighAdmin maintenance and the SQL Server Agent executes the same procedure every minute.
- Reversal/cancellation is guarded by ownership, transaction state, and minimum-balance consequences.
- Account balance posting fires branch-ledger automation; installment-payment transaction status fires installment synchronization.

### 9.3 TransactionStatus effects

| Status    | Balance effect                                                           | Workflow effect                                             |
|-----------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| Pending   | No posted balance movement; outgoing amount is held in available balance | Waits until ReadyToCompleteAt and finalizer/batch.          |
| Completed | Debit/credit posted; CompletedAt required                                | Ledger and installment triggers react.                      |
| Failed    | No completed posting                                                     | Linked installment becomes retryable.                       |
| Cancelled | Reversal/cancellation semantics                                          | Cannot be processed again as Pending; audit records reason. |

### 9.4 Readiness tiers

| Amount                  | Delay       | Operational meaning                                   |
|-------------------------|-------------|-------------------------------------------------------|
| Below 5,000             | 0 seconds   | Eligible immediately for the next processing attempt. |
| 5,000 to below 25,000   | 60 seconds  | Short hold.                                           |
| 25,000 to below 100,000 | 180 seconds | Medium hold.                                          |
| 100,000 or more         | 300 seconds | Longest configured hold.                              |

### 9.5 Procedures that enforce or expose these rules

#### dbo.sp\_Transaction\_Deposit

**Purpose.** Initiates an owner-authorized deposit.

**Authorization/scope.** Only the account owner; staff roles do not grant transaction authority.

**API use.** POST/api/transactions/deposit

**Rule behavior.** Validates caller/customer ownership, positive amount, destination state, transaction type, computes ReadyToCompleteAt, inserts Pending transaction, and audits DepositInitiated.

**Data effects.** Reads: dbo.Account, dbo.TransactionType, dbo.Users. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The Process Pending Bank Transactions Agent finalizes it when ready.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Desposit\_SP.sql

**dbo.sp\_Transaction\_Withdrawal**

**Purpose.** Initiates an owner-authorized withdrawal.

**Authorization/scope.** Only the account owner.

**API use.** POST/api/transactions/withdraw

**Rule behavior.** Locks account and pending holds, requires Active source, positive amount, and enough available balance to preserve AccountType.MinBalance; inserts Pending withdrawal and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent finalizes it; until then the amount is reserved in available balance.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Withdraw\_SP.sql

**dbo.sp\_Transaction\_Transfer**

**Purpose.** Initiates an owner-authorized transfer.

**Authorization/scope.** Only the source-account owner; destination ownership is not required.

**API use.** POST/api/transactions/transfer

**Rule behavior.** Validates distinct accounts, source Active, destination not Closed/Frozen, source ownership, available funds/minimum balance, computes delay, inserts Pending transfer, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent posts both account balance changes when ReadyToCompleteAt is reached.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Transfer\_SP.sql

**dbo.sp\_Transaction\_Finalize**

**Purpose.** Posts or fails one pending transaction.

**Authorization/scope.** Owner for manual endpoint, or trusted internal batch behavior.

**API use.** POST/api/transactions/:transactionID/finalize

**Rule behavior.** Locks the transaction and affected accounts, requires Pending and ready state, applies deposit/withdrawal/transfer/interest balance effects, marks Completed or Failed, writes CompletedAt, reactivates Dormant incoming accounts when applicable, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Customer, dbo.Transactions, dbo.Users. Writes: dbo.Account, dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The batch procedure uses equivalent logic for Agent-driven processing. Account balance changes fire the branch-ledger trigger; transaction status changes fire the installment trigger.



**Source.** database/scripts/StoredProcedures/TransactionProcedures/TransactionFinalizer.sql

#### dbo.sp\_Transaction\_ProcessPendingBatch

**Purpose.** Finalizes all ready pending transactions as one scheduled maintenance sweep.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** POST/api/transactions/maintenance/process-pending-batch

**Rule behavior.** Selects ready Pending rows with update locks, processes each type set-wise, marks Completed/Failed, updates accounts, and writes audit entries.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Transactions. Writes: dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** Called every minute by Process Pending Bank Transactions.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Procedure\_ProcessPendingBatch.sql

#### dbo.sp\_Transaction\_Reverse

**Purpose.** Cancels a pending transaction or reverses a completed one.

**Authorization/scope.** Only the transaction account owner under the current policy.

**API use.** POST/api/transactions/:transactionID/reverse

**Rule behavior.** Locks transaction/accounts, permits only Pending or Completed, reverses posted balance effects while protecting minimum balance, handles linked installment payment state, marks Cancelled, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Customer, dbo.TransactionType, dbo.Transactions, dbo.Users. Writes: dbo.Account, dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job reverses transactions. Trigger effects occur when account balances or transaction statuses change.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Procedure\_ReverseTransaction.sql

#### dbo.sp\_Transaction\_GetAccountHistory

**Purpose.** Returns newest-first transaction history for one authorized account.

**Authorization/scope.** Self, current branch, or all depending on effective role and requested scope.

**API use.** GET/api/accounts/:accountID/history

**Rule behavior.** Validates date range and account visibility, then returns debit and credit participation with transaction timestamps/status.

**Data effects.** Reads: dbo.Account, dbo.Users, dbo.vw\_AccountTransactionHistory.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The pending transaction Agent changes Pending rows to Complet-

ed/Failed, which is reflected in subsequent history reads.

**Source.** database/scripts/StoredProcedures/AccountProcedures/Procedure\_AccountHistory.sql

## 9.6 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Transactions\_EnforceTypePattern, dbo.TR\_Transactions\_SyncInstallmentStatus.

**Relevant SQL Server Agent jobs:** Process Pending Bank Transactions.

## Chapter 10

### Loan

Loan stores the contract-level borrower, branch, amount, rate, dates, and overall status. Payment is deliberately routed through Transactions rather than directly changing account balances.

#### 10.1 Attributes and structural constraints

##### Attributes of `dbo.Loan`

| Attribute    | Type          | Constraint/default                   | Application meaning             |
|--------------|---------------|--------------------------------------|---------------------------------|
| LoanID       | INTIDENTITY   | PK, not null                         | Loan identifier.                |
| CustomerID   | INT           | FK Customer                          | Borrower.                       |
| BranchID     | INT           | FK Branch                            | Originating/operational branch. |
| LoanAmount   | DECIMAL(18,2) | Positive                             | Principal.                      |
| InterestRate | DECIMAL(5,2)  | Not null                             | Loan rate.                      |
| StartDate    | DATE          | Not null                             | Start date.                     |
| EndDate      | DATE          | Nullable, not earlier than StartDate | Contract end.                   |
| LoanStatus   | NVARCHAR(20)  | Active/Paid/Defaulted                | Contract state.                 |

#### 10.2 Entity rules

- LoanAmount must be positive; status is Active, Paid, or Defaulted.
- Employee/Admin may create loans only in current branch; HighAdmin may choose any valid branch.
- Borrower/customer must exist and be active.
- Loan creation generates the full installment schedule in one transaction.
- Customer personal scope shows own loans; staff operational scope shows current-branch loans; HighAdmin shows all.
- Only the borrower can pay an installment and the paying account must also belong to that borrower.
- Loan becomes Paid automatically when every installment is Paid.
- Loan becomes Defaulted when overdue processing marks at least one installment Defaulted.
- Branch manager manual overdue processing is restricted to current branch; HighAdmin can process all or a selected branch; Agent uses trusted all-branch mode.

#### 10.3 LoanStatus effects

| Status | Effect                                                                                        |
|--------|-----------------------------------------------------------------------------------------------|
| Active | Schedule has unpaid installments; eligible Pending/Late installments may be paid by borrower. |

| Status    | Effect                                                                                                                              |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| Paid      | Every installment is Paid; set automatically by trigger after payment transaction completion.                                       |
| Defaulted | At least one installment is Defaulted; appears in overdue/default reporting and normal payment of Defaulted installment is blocked. |

## 10.4 Visibility and payment

Staff may inspect current-branch loans and schedules, HighAdmin all loans, and every authenticated role has a separate personal-loan scope. Viewing does not grant payment authority: only the borrower can pay, using a borrower-owned Active account.

## 10.5 Procedures that enforce or expose these rules

### `dbo.sp_Loan_Create`

**Purpose.** Creates a loan and complete installment schedule.

**Authorization/scope.** Employee/Admin in current branch; HighAdmin may choose any branch.

**API use.** POST/api/loans

**Rule behavior.** Validates active customer, branch, amount, rate, count, and dates; inserts Loan and evenly calculated Installment rows in one transaction; audits LoanCreated.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`. Writes: `dbo.AuditLog`, `dbo.Installment`, `dbo.Loan`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent creates loans. The overdue Agent later monitors the generated schedule.

**Source.** `database/scripts/StoredProcedures/LoanProcedures/Procedure_Loan_Create.sql`

### `dbo.sp_Loan_GetStatus`

**Purpose.** Returns loan summary and installment schedule.

**Authorization/scope.** Borrower sees own; staff visibility is prevalidated to current branch; HighAdmin sees all.

**API use.** GET/api/loans/:loanID/status

**Rule behavior.** Returns separate recordsets for loan and installments, including payment transaction readiness and dates. SQL independently prevents customer cross-loan access.

**Data effects.** Reads: `dbo.Customer`, `dbo.Installment`, `dbo.Loan`, `dbo.Transactions`, `dbo.Users`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The payment and overdue jobs change statuses later reflected here.

**Source.** `database/scripts/StoredProcedures/LoanProcedures/Procedure_Loan_GetStatus.sql`

**dbo.sp\_Loan\_PayInstallment**

**Purpose.** Initiates payment of one installment using a borrower-owned account.

**Authorization/scope.** Borrower only; the payment account and loan customer must match the authenticated CustomerID.

**API use.** POST/api/loans/installments/:installmentID/pay

**Rule behavior.** Locks installment/loan/account, rejects Paid/Defaulted or duplicate in-flight payment, calls withdrawal logic, stores PaymentTransactionID, and audits the initiated payment.

**Data effects.** Reads: dbo.Account, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog, dbo.Installment.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending transaction Agent finalizes the withdrawal. TR\_Transactions\_SyncInstallmentStatus then marks the installment Paid or releases a failed payment link.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_PayInstallment.sql

**dbo.sp\_Loan\_ProcessOverdueInstallments**

**Purpose.** Marks overdue installments Late or Defaulted and defaults loans where required.

**Authorization/scope.** Manual branch manager/Admin current branch; HighAdmin all or selected branch; Agent all branches.

**API use.** POST/api/loans/maintenance/process-overdue-installments

**Rule behavior.** Resolves manual scope when UserID is supplied, ignores installments with active in-flight payments, marks past-due rows Late and rows over 90 days Defaulted, updates affected loans, and audits the sweep.

**Data effects.** Reads: dbo.EMPB, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called daily at 00:00 by Process Overdue Loan Installments. The Agent invokes it without UserID/BranchID, which is the trusted all-branch mode.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_ProcessOverdueInstallments.sql

## 10.6 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Transactions\_SyncInstallmentStatus.

**Relevant SQL Server Agent jobs:** Process Overdue Loan Installments.

## Chapter 11

# Installment

Installment stores the payment schedule. PaymentTransactionID creates the bridge from a scheduled obligation to the normal transaction-processing system.

### 11.1 Attributes and structural constraints

#### Attributes of dbo.Installment

| Attribute            | Type          | Constraint/default             | Application meaning                                  |
|----------------------|---------------|--------------------------------|------------------------------------------------------|
| InstallmentID        | INTIDENTITY   | PK, not null                   | Installment identifier.                              |
| LoanID               | INT           | FK Loan                        | Parent loan.                                         |
| DueDate              | DATE          | Not null                       | Due date.                                            |
| Amount               | DECIMAL(18,2) | Positive                       | Required payment.                                    |
| PaidDate             | DATE          | Nullable                       | Date set when linked payment completes.              |
| InstallmentStatus    | NVARCHAR(20)  | Pending, Paid, Late, Defaulted | Payment state.                                       |
| PaymentTransactionID | INT           | Nullable FK Transactions       | One in-flight or completed payment transaction link. |

### 11.2 Entity rules

- Amount must be positive; status is Pending, Paid, Late, or Defaulted.
- PaymentTransactionID provides a single in-flight/completed payment link and prevents duplicate payment attempts.
- Pending is payable before due date; Late is payable after due date but before default threshold; Paid is terminal for normal payment; Defaulted blocks normal payment.
- Overdue processing ignores rows with an active in-flight payment transaction.
- More than 90 days overdue becomes Defaulted; past due but within threshold becomes Late.
- When linked transaction becomes Completed, trigger marks Paid and sets PaidDate.
- When linked transaction becomes Failed, trigger clears PaymentTransactionID and restores Pending or Late so retry is possible.
- Payment uses the normal withdrawal transaction path, preserving balance, minimum-balance, audit, and ledger rules.

### 11.3 InstallmentStatus effects

| Status    | Payment eligibility          | Transition rule                                                       |
|-----------|------------------------------|-----------------------------------------------------------------------|
| Pending   | Payable if no in-flight link | Becomes Late after due date, Paid after completed linked transaction. |
| Late      | Payable if no in-flight link | Becomes Defaulted after threshold or Paid after completion.           |
| Defaulted | Normal payment blocked       | Set by overdue processing after more than 90 days.                    |
| Paid      | Not payable again            | Trigger sets PaidDate when linked transaction completes.              |

## 11.4 Procedures that enforce or expose these rules

### dbo.sp\_Loan\_Create

**Purpose.** Creates a loan and complete installment schedule.

**Authorization/scope.** Employee/Admin in current branch; HighAdmin may choose any branch.

**API use.** POST/api/loans

**Rule behavior.** Validates active customer, branch, amount, rate, count, and dates; inserts Loan and evenly calculated Installment rows in one transaction; audits LoanCreated.

**Data effects.** Reads: dbo.Branch, dbo.Customer. Writes: dbo.AuditLog, dbo.Installment, dbo.Loan.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent creates loans. The overdue Agent later monitors the generated schedule.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_Create.sql

### dbo.sp\_Loan\_GetStatus

**Purpose.** Returns loan summary and installment schedule.

**Authorization/scope.** Borrower sees own; staff visibility is prevalidated to current branch; HighAdmin sees all.

**API use.** GET/api/loans/:loanID/status

**Rule behavior.** Returns separate recordsets for loan and installments, including payment transaction readiness and dates. SQL independently prevents customer cross-loan access.

**Data effects.** Reads: dbo.Customer, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The payment and overdue jobs change statuses later reflected here.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_GetStatus.sql

### dbo.sp\_Loan\_PayInstallment

**Purpose.** Initiates payment of one installment using a borrower-owned account.

**Authorization/scope.** Borrower only; the payment account and loan customer must match

the authenticated CustomerID.

**API use.** POST/api/loans/installments/:installmentID/pay

**Rule behavior.** Locks installment/loan/account, rejects Paid/Defaulted or duplicate in-flight payment, calls withdrawal logic, stores PaymentTransactionID, and audits the initiated payment.

**Data effects.** Reads: dbo.Account, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog, dbo.Installment.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending transaction Agent finalizes the withdrawal. TR\_Transactions\_SyncInstallmentStatus then marks the installment Paid or releases a failed payment link.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_PayInstallment.sql

#### dbo.sp\_Loan\_ProcessOverdueInstallments

**Purpose.** Marks overdue installments Late or Defaulted and defaults loans where required.

**Authorization/scope.** Manual branch manager/Admin current branch; HighAdmin all or selected branch; Agent all branches.

**API use.** POST/api/loans/maintenance/process-overdue-installments

**Rule behavior.** Resolves manual scope when UserID is supplied, ignores installments with active in-flight payments, marks past-due rows Late and rows over 90 days Defaulted, updates affected loans, and audits the sweep.

**Data effects.** Reads: dbo.EMPB, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called daily at 00:00 by Process Overdue Loan Installments. The Agent invokes it without UserID/BranchID, which is the trusted all-branch mode.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_ProcessOverdueInstallments.sql

## 11.5 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_Transactions\_SyncInstallmentStatus.

**Relevant SQL Server Agent jobs:** Process Overdue Loan Installments, Process Pending Bank Transactions.



## Chapter 12

# BranchLedger

BranchLedger is an automatically maintained history of branch-level balance movements. It is not edited by report endpoints.

### 12.1 Attributes and structural constraints

#### Attributes of `dbo.BranchLedger`

| Attribute      | Type          | Constraint/default       | Application meaning                            |
|----------------|---------------|--------------------------|------------------------------------------------|
| BranchLedgerID | INTIDENTITY   | PK, not null             | Ledger row.                                    |
| BranchID       | INT           | FK Branch                | Affected branch.                               |
| TransactionID  | INT           | Nullable FK Transactions | Optional causal transaction.                   |
| DeltaAmount    | DECIMAL(18,2) | Not null                 | Signed branch-balance movement.                |
| BalanceAfter   | DECIMAL(18,2) | Not null                 | Branch cached balance after the trigger batch. |
| EntryDate      | DATETIME      | Default current time     | Ledger timestamp.                              |
| Description    | NVARCHAR(200) | Nullable                 | Cause/context.                                 |

### 12.2 Entity rules

- Rows are inserted automatically when `Account.Balance` changes; report reads do not update the ledger.
- `DeltaAmount` is signed; `BalanceAfter` is the branch cached balance after the trigger batch.
- Cross-branch transfer creates a negative source-branch movement and positive destination-branch movement.
- Same-branch transfer has net zero branch balance, though account-level delta rows may both appear.
- `TransactionID` may be `NULL` for trigger-generated rows because the `Account` trigger does not automatically receive causal transaction context.
- For a multi-account statement, rows for one branch can share the same post-batch `BalanceAfter`; this is a documented set-based simplification.
- Trigger execution is atomic with the account update: failure rolls back the originating balance change.

### 12.3 Trigger update algorithm

For every `Account INSERT/UPDATE` whose `Balance` changed, the trigger calculates  $\Delta = \text{new Balance} - \text{old Balance}$ , aggregates the net delta per branch, updates `Branch.Balance`, and inserts one `BranchLedger` row per changed account. The report view is read-only.

**Set-based simplification**

When one statement changes several accounts in one branch, all generated rows can show the same BalanceAfter: the branch balance after the complete batch. This is intentional and keeps the trigger set based.

## 12.4 Triggers and scheduled automation

**Relevant triggers:** `dbo.TR_Account_SyncBranchLedger`.

## Chapter 13

# Roles

Roles is the application-role catalogue, not SQL Server database roles. Role assignment and role effectiveness are separate concepts.

### 13.1 Attributes and structural constraints

#### Attributes of `dbo.Roles`

| Attribute | Type         | Constraint/default | Application meaning                   |
|-----------|--------------|--------------------|---------------------------------------|
| RoleID    | INTIDENTITY  | PK, not null       | Role identifier.                      |
| RoleName  | NVARCHAR(50) | Unique, not null   | Customer, Employee, Admin, HighAdmin. |

### 13.2 Entity rules

- Role catalogue is seeded with Customer, Employee, Admin, HighAdmin.
- Assigned role is not automatically effective; current user/customer/employee/assignment state is recalculated.
- Employee/Admin users must also have Customer role.
- HighAdmin must also have Customer role, must not have EmployeeID, and cannot be combined with Employee/Admin.
- Only one active HighAdmin user is allowed.
- Admin is reserved for Active Branch Manager/Vice Manager with CanAccessAdmin=1 and an effective Employee role.

### 13.3 Assigned versus effective roles

| Role      | Assignment requirement                      | Effective requirement                                                 |
|-----------|---------------------------------------------|-----------------------------------------------------------------------|
| Customer  | CustomerID and Customer role row            | User and Customer active.                                             |
| Employee  | EmployeeID, Customer role, Employee role    | Employee active.                                                      |
| Admin     | Customer+Employee+Admin role rows           | Active Branch Manager/Vice Manager, CanAccessAdmin=1, current branch. |
| HighAdmin | Customer+HighAdmin role rows, no EmployeeID | Active user/customer; no Employee/Admin combination.                  |

## 13.4 Procedures that enforce or expose these rules

### dbo.sp\_User\_SignUp

**Purpose.** Creates a self-service customer login and guarantees the Customer role.

**Authorization/scope.** Public endpoint; uniqueness, active-customer, and user-link rules remain enforced in SQL.

**API use.** POST/api/auth/signup

**Rule behavior.** Creates or reuses the customer identity, hashes the password with the project hash function, inserts the Users row, assigns the Customer role, and records UserSignUp.

**Data effects.** Reads: dbo.Roles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job is involved. Signup is synchronous and atomic.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_SignUp\_UPDATED.sql

### dbo.sp\_Employee\_CreateUserAccount

**Purpose.** Creates an application login for an existing ordinary employee.

**Authorization/scope.** Scoped Admin or HighAdmin, with branch and manager-level restrictions.

**API use.** POST/api/employees/:employeeID/create-user-account

**Rule behavior.** Creates/reuses Customer, creates/reuses Users, links EmployeeID/CustomerID, assigns Customer and Employee roles, and audits EmployeeUserAccountCreated.

**Data effects.** Reads: dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_CreateUserAccount.sql

### dbo.sp\_HighAdmin\_CreateInitialUser

**Purpose.** Creates the first HighAdmin identity exactly once.

**Authorization/scope.** One-time setup endpoint; procedure refuses creation when a HighAdmin already exists.

**API use.** POST/api/setup/initial-high-admin

**Rule behavior.** Creates or reuses an active customer, creates/reuses a user, assigns Customer and HighAdmin, and audits HighAdminCreated in one transaction.

**Data effects.** Reads: dbo.Customer, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job is involved.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_HighAdmin\_C

```
reateInitialUser.sql
```

#### dbo.sp\_HighAdmin\_HireManager

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_HireManager.sql

#### dbo.sp\_HighAdmin\_PromoteToManager

**Purpose.** Promotes an active employee with login to manager level.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/promote

**Rule behavior.** Validates target and branch, updates title/admin capability, adjusts assignment when necessary, ensures Admin role, and audits EmployeePromotedToManager.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_PromoteToManager.sql

#### dbo.sp\_HighAdmin\_DowngradeManager

**Purpose.** Removes manager status and returns the employee to an ordinary title.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/downgrade

**Rule behavior.** Validates a non-manager replacement title, updates JobTitle/CanAccessAdmin, removes effective Admin behavior through triggers/role rules, and audits ManagerDowngraded.

**Data effects.** Reads: dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_DowngradeManager.sql

## 13.5 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_UserRoles\_EnforceAccessModel.

## Chapter 14

### Users

Users stores application login identities. It links every login to a Customer and staff logins optionally to an Employee.

#### 14.1 Attributes and structural constraints

##### Attributes of `dbo.Users`

| Attribute    | Type         | Constraint/default                              | Application meaning                |
|--------------|--------------|-------------------------------------------------|------------------------------------|
| UserID       | INTIDENTITY  | PK, not null                                    | Application login identity.        |
| Username     | NVARCHAR(50) | Unique, not null                                | Login name.                        |
| PasswordHash | CHAR(64)     | Not null                                        | SHA-256 hexadecimal password hash. |
| EmployeeID   | INT          | Nullable unique FK Employee                     | Staff link.                        |
| CustomerID   | INT          | Required by access triggers; unique FK Customer | Customer identity link.            |
| IsActive     | BIT          | Default 1                                       | Login eligibility.                 |

#### 14.2 Entity rules

- Username is unique; password plaintext is never stored in Users.
- Every application user must have CustomerID and the linked customer must be active.
- At most one login may link to one Customer and at most one login may link to one Employee.
- Employee/Admin roles require EmployeeID; HighAdmin forbids EmployeeID.
- Users.IsActive controls whether login is possible; Employee.EmpStatus separately controls staff privilege effectiveness.
- Trusted UserID comes from session validation and overrides any client-supplied identity value in route inputs.

#### 14.3 Identity combinations

| User kind      | Links                       | Meaning                                                     |
|----------------|-----------------------------|-------------------------------------------------------------|
| Customer-only  | CustomerID, EmployeeID NULL | Personal banking identity.                                  |
| Employee/Admin | CustomerID and EmployeeID   | Staff identity plus personal banking identity.              |
| HighAdmin      | CustomerID, EmployeeID NULL | Central governance identity with personal banking identity. |

## 14.4 Procedures that enforce or expose these rules

### dbo.sp\_User\_SignUp

**Purpose.** Creates a self-service customer login and guarantees the Customer role.

**Authorization/scope.** Public endpoint; uniqueness, active-customer, and user-link rules remain enforced in SQL.

**API use.** POST/api/auth/signup

**Rule behavior.** Creates or reuses the customer identity, hashes the password with the project hash function, inserts the Users row, assigns the Customer role, and records UserSignUp.

**Data effects.** Reads: dbo.Roles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job is involved. Signup is synchronous and atomic.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_SignUp\_UPDATED.sql

### dbo.sp\_User\_Login

**Purpose.** Authenticates one supported login identifier and issues an opaque server-side session token.

**Authorization/scope.** Public endpoint protected by the backend login rate limiter. Effective roles are recalculated rather than trusting assigned roles alone.

**API use.** POST/api/auth/login

**Rule behavior.** Validates the password hash, confirms active user/customer state, determines effective Customer/Employee/Admin/HighAdmin roles, inserts Sessions with a fixed ExpiresAt, and returns identity plus role context.

**Data effects.** Reads: dbo.Customer, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job expires sessions. Expiration is checked on every protected request by sp\_User\_ValidateSession.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_Login.sql

### dbo.sp\_User\_ValidateSession

**Purpose.** Validates a bearer token and constructs the authoritative session context used as req.user.

**Authorization/scope.** Every protected request. It may additionally check a RequiredRole parameter.

**API use.** Allprotected/apiroutesthroughauthenticate()

**Rule behavior.** Finds the active session, rejects expired tokens, deactivates expired rows, recalculates roles and current branch, and returns UserID, CustomerID, EmployeeID, roles, and branch context.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles,



`dbo.Sessions, dbo.UserRoles, dbo.Users`. Writes: `dbo.AuditLog, dbo.Sessions`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No scheduled cleanup is required for correctness because validation is request-time and authoritative.

**Source.** `database/scripts/StoredProcedures/Session/Procedure_User_ValidateSession.sql`

#### `dbo.sp_User_Logout`

**Purpose.** Revokes one session token.

**Authorization/scope.** Authenticated caller owning the presented bearer token.

**API use.** `POST/api/auth/logout`

**Rule behavior.** Marks `Sessions.IsActive=0`, sets `LogoutTime`, and records `UserLogout`.

**Data effects.** Reads: `dbo.Sessions`. Writes: `dbo.AuditLog, dbo.Sessions`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation. Lock hints used: `HOLDLOCK, UPDLOCK`.

**Automation relationship.** No Agent job is involved.

**Source.** `database/scripts/StoredProcedures/UserProcedures/Procedure_User_Logout.sql`

#### `dbo.sp_HighAdmin_CreateInitialUser`

**Purpose.** Creates the first HighAdmin identity exactly once.

**Authorization/scope.** One-time setup endpoint; procedure refuses creation when a HighAdmin already exists.

**API use.** `POST/api/setup/initial-high-admin`

**Rule behavior.** Creates or reuses an active customer, creates/reuses a user, assigns Customer and HighAdmin, and audits `HighAdminCreated` in one transaction.

**Data effects.** Reads: `dbo.Customer, dbo.Roles, dbo.UserRoles, dbo.Users`. Writes: `dbo.AuditLog, dbo.UserRoles, dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK, UPDLOCK`.

**Automation relationship.** No Agent job is involved.

**Source.** `database/scripts/StoredProcedures/UserProcedures/Procedure_HighAdmin_CreateInitialUser.sql`

#### `dbo.sp_Employee_CreateUserAccount`

**Purpose.** Creates an application login for an existing ordinary employee.

**Authorization/scope.** Scoped Admin or HighAdmin, with branch and manager-level restrictions.

**API use.** `POST/api/employees/:employeeID/create-user-account`

**Rule behavior.** Creates/reuses Customer, creates/reuses Users, links `EmployeeID/CustomerID`, assigns Customer and Employee roles, and audits `EmployeeUserAccountCreated`.

**Data effects.** Reads: `dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles,`

`dbo.Users`. Writes: `dbo.AuditLog`, `dbo.UserRoles`, `dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_Employee_CreateUserAccount.sql`

### `dbo.sp_HighAdmin_HireManager`

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** `POST/api/highadmin/managers`

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: `dbo.Branch`, `dbo.Customer`, `dbo.EMPB`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.Employee`, `dbo.UserRoles`, `dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/Employees/Procedure_HighAdmin_HireManager.sql`

## 14.5 Triggers and scheduled automation

**Relevant triggers:** `dbo.TR_Users_EnforceUserLinks`.

## Chapter 15

# UserRoles

UserRoles assigns application roles while triggers enforce valid combinations and hierarchy.

### 15.1 Attributes and structural constraints

#### Attributes of `dbo.UserRoles`

| Attribute     | Type                 | Constraint/default           | Application meaning    |
|---------------|----------------------|------------------------------|------------------------|
| UserRoleID    | INTIDENTITY          | PK, not null                 | Assignment row.        |
| UserID        | INT                  | FK Users, cascade delete     | User.                  |
| RoleID        | INT                  | FK Roles, cascade delete     | Assigned role.         |
| UserID+RoleID | Composite uniqueness | No duplicate role assignment | Assignment uniqueness. |

### 15.2 Entity rules

- UserID+RoleID is unique, preventing duplicate assignments.
- Customer role requires CustomerID.
- Employee/Admin roles require EmployeeID and Customer role.
- Admin assignment requires an Active Branch Manager/Vice Manager with CanAccessAdmin=1.
- HighAdmin requires CustomerID, forbids EmployeeID, requires Customer role, and is mutually exclusive with Employee/Admin.
- Role triggers block invalid combinations even when inserted outside normal procedures.
- Effective-role calculation is performed by `fn_UserHasEffectiveRole` and session procedures using current state.

### 15.3 Role-combination rules

The triggers enforce that Employee/Admin users also have Customer role; Admin is manager-only; HighAdmin also has Customer role but cannot have Employee/Admin; and only one active HighAdmin exists. These checks apply even if a direct INSERT attempts to bypass a stored procedure.

### 15.4 Procedures that enforce or expose these rules

#### `dbo.sp_User_SignUp`

**Purpose.** Creates a self-service customer login and guarantees the Customer role.

**Authorization/scope.** Public endpoint; uniqueness, active-customer, and user-link rules remain enforced in SQL.

**API use.** POST/api/auth/signup

**Rule behavior.** Creates or reuses the customer identity, hashes the password with the project hash function, inserts the Users row, assigns the Customer role, and records UserSignUp.

**Data effects.** Reads: dbo.Roles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job is involved. Signup is synchronous and atomic.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_SignUp\_UPDATED.sql

### dbo.sp\_User\_Login

**Purpose.** Authenticates one supported login identifier and issues an opaque server-side session token.

**Authorization/scope.** Public endpoint protected by the backend login rate limiter. Effective roles are recalculated rather than trusting assigned roles alone.

**API use.** POST/api/auth/login

**Rule behavior.** Validates the password hash, confirms active user/customer state, determines effective Customer/Employee/Admin/HighAdmin roles, inserts Sessions with a fixed ExpiresAt, and returns identity plus role context.

**Data effects.** Reads: dbo.Customer, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job expires sessions. Expiration is checked on every protected request by sp\_User\_ValidateSession.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_Login.sql

### dbo.sp\_User\_ValidateSession

**Purpose.** Validates a bearer token and constructs the authoritative session context used as req.user.

**Authorization/scope.** Every protected request. It may additionally check a RequiredRole parameter.

**API use.** Allprotected/apiroutesthroughauthenticate()

**Rule behavior.** Finds the active session, rejects expired tokens, deactivates expired rows, recalculates roles and current branch, and returns UserID, CustomerID, EmployeeID, roles, and branch context.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.Sessions, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No scheduled cleanup is required for correctness because validation is request-time and authoritative.

**Source.** database/scripts/StoredProcedures/Session/Procedure\_User\_ValidateSessi

on.sql

#### dbo.sp\_HighAdmin\_CreateInitialUser

**Purpose.** Creates the first HighAdmin identity exactly once.

**Authorization/scope.** One-time setup endpoint; procedure refuses creation when a HighAdmin already exists.

**API use.** POST/api/setup/initial-high-admin

**Rule behavior.** Creates or reuses an active customer, creates/reuses a user, assigns Customer and HighAdmin, and audits HighAdminCreated in one transaction.

**Data effects.** Reads: dbo.Customer, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job is involved.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_HighAdmin\_CreateInitialUser.sql

#### dbo.sp\_Employee\_CreateUserAccount

**Purpose.** Creates an application login for an existing ordinary employee.

**Authorization/scope.** Scoped Admin or HighAdmin, with branch and manager-level restrictions.

**API use.** POST/api/employees/:employeeID/create-user-account

**Rule behavior.** Creates/reuses Customer, creates/reuses Users, links EmployeeID/CustomerID, assigns Customer and Employee roles, and audits EmployeeUserAccountCreated.

**Data effects.** Reads: dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_CreateUserAccount.sql

#### dbo.sp\_HighAdmin\_HireManager

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK,

UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_HireManager.sql

#### dbo.sp\_HighAdmin\_PromoteToManager

**Purpose.** Promotes an active employee with login to manager level.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/promote

**Rule behavior.** Validates target and branch, updates title/admin capability, adjusts assignment when necessary, ensures Admin role, and audits EmployeePromotedToManager.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_PromoteToManager.sql

#### dbo.sp\_HighAdmin\_DowngradeManager

**Purpose.** Removes manager status and returns the employee to an ordinary title.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/downgrade

**Rule behavior.** Validates a non-manager replacement title, updates JobTitle/CanAccessAdmin, removes effective Admin behavior through triggers/role rules, and audits ManagerDowngraded.

**Data effects.** Reads: dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_DowngradeManager.sql

## 15.5 Triggers and scheduled automation

**Relevant triggers:** dbo.TR\_UserRoles\_EnforceAccessModel.

## Chapter 16

### Sessions

Sessions stores server-side bearer sessions. The token is only a lookup key; identity and privileges are recalculated on each request.

#### 16.1 Attributes and structural constraints

##### Attributes of `dbo.Sessions`

| Attribute    | Type          | Constraint/default             | Application meaning          |
|--------------|---------------|--------------------------------|------------------------------|
| SessionID    | INTIDENTITY   | PK, not null                   | Session row.                 |
| UserID       | INT           | FK Users, cascade delete       | Authenticated user.          |
| SessionToken | NVARCHAR(200) | Unique, not null               | Opaque bearer token.         |
| LoginTime    | DATETIME      | Default current time           | Start.                       |
| ExpiresAt    | DATETIME      | Default LoginTime + 10 minutes | Fixed absolute expiry.       |
| LogoutTime   | DATETIME      | Nullable                       | Explicit or expiration time. |
| IsActive     | BIT           | Default 1                      | Revocation state.            |

#### 16.2 Entity rules

- SessionToken is opaque; it does not encode UserID, role, or branch.
- Login creates a fixed absolute ExpiresAt using `SESSION_TTL_MINUTES`, configured as 10 minutes.
- Validation requires IsActive=1 and ExpiresAt greater than SQL Server current time.
- Expired validation deactivates the row, records LogoutTime, audits expiration, and causes HTTP 401.
- Logout deactivates only the presented token.
- Every protected request calls `sp_User_ValidateSession`; `req.user` is reconstructed from database state.
- Role and branch changes take effect on the next request without reissuing the token.

#### 16.3 Fixed ten-minute lifecycle

1. Login validates credentials and inserts SessionToken, LoginTime, ExpiresAt, and IsActive=1.
  2. The frontend sends **Authorization: Bearer <token>** on protected calls.
  3. `authenticate()` calls `sp_User_ValidateSession`.
  4. Validation rejects revoked or expired rows, recalculates roles and branch, and returns the authoritative identity.
  5. Logout or expiration sets IsActive=0 and LogoutTime.
- The ten-minute lifetime is absolute, not sliding: normal activity does not extend ExpiresAt.

## 16.4 Procedures that enforce or expose these rules

### dbo.sp\_User\_Login

**Purpose.** Authenticates one supported login identifier and issues an opaque server-side session token.

**Authorization/scope.** Public endpoint protected by the backend login rate limiter. Effective roles are recalculated rather than trusting assigned roles alone.

**API use.** POST/api/auth/login

**Rule behavior.** Validates the password hash, confirms active user/customer state, determines effective Customer/Employee/Admin/HighAdmin roles, inserts Sessions with a fixed ExpiresAt, and returns identity plus role context.

**Data effects.** Reads: dbo.Customer, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job expires sessions. Expiration is checked on every protected request by sp\_User\_ValidateSession.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_Login.sql

### dbo.sp\_User\_ValidateSession

**Purpose.** Validates a bearer token and constructs the authoritative session context used as req.user.

**Authorization/scope.** Every protected request. It may additionally check a RequiredRole parameter.

**API use.** Allprotected/apiroutesthroughauthenticate()

**Rule behavior.** Finds the active session, rejects expired tokens, deactivates expired rows, recalculates roles and current branch, and returns UserID, CustomerID, EmployeeID, roles, and branch context.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.Sessions, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No scheduled cleanup is required for correctness because validation is request-time and authoritative.

**Source.** database/scripts/StoredProcedures/Session/Procedure\_User\_ValidateSession.sql

### dbo.sp\_User\_Logout

**Purpose.** Revokes one session token.

**Authorization/scope.** Authenticated caller owning the presented bearer token.

**API use.** POST/api/auth/logout

**Rule behavior.** Marks Sessions.IsActive=0, sets LogoutTime, and records UserLogout.

**Data effects.** Reads: dbo.Sessions. Writes: dbo.AuditLog, dbo.Sessions.



**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job is involved.

**Source.** `database/scripts/StoredProcedures/UserProcedures/Procedure_User_Logout.sql`

## Chapter 17

# AuditLog

AuditLog records security and business events without becoming the source of truth for account balances or workflow state.

### 17.1 Attributes and structural constraints

#### Attributes of `dbo.AuditLog`

| Attribute  | Type          | Constraint/default   | Application meaning             |
|------------|---------------|----------------------|---------------------------------|
| AuditID    | INTIDENTITY   | PK, not null         | Audit row.                      |
| UserID     | INT           | Nullable FK Users    | Actor when known.               |
| ActionType | NVARCHAR(100) | Not null             | Action category.                |
| TableName  | NVARCHAR(100) | Nullable             | Affected entity.                |
| RecordID   | INT           | Nullable             | Affected row.                   |
| ActionDate | DATETIME      | Default current time | Action timestamp.               |
| Details    | NVARCHAR(MAX) | Nullable             | Non-secret operational details. |

### 17.2 Entity rules

- Audit records capture actor, action type, entity/table, record ID, timestamp, and details.
- Passwords, password hashes, and raw session tokens must not be placed in Details.
- Operational procedures write audit rows for creation, viewing/searching where configured, lifecycle changes, authentication, maintenance, and governance.
- SQL Server Agent executions may have no application UserID; this distinguishes system maintenance from user actions.
- Audit reporting is exposed only through the safe audit view and dedicated restricted database pool.
- Routine UI searches can legitimately produce audit noise; report refresh itself is read-only unless the page also performs an audited operational query.

### 17.3 Audit and report behavior

The audit trail reports records newest first. Details can include filters and access mode, such as `IncludeClosed=1`, but must not contain secrets. Viewing the audit report does not itself update Account or BranchLedger. Some home-page operational lookups are audited because they call audited search procedures.

## 17.4 Procedures that enforce or expose these rules

### dbo.sp\_User\_SignUp

**Purpose.** Creates a self-service customer login and guarantees the Customer role.

**Authorization/scope.** Public endpoint; uniqueness, active-customer, and user-link rules remain enforced in SQL.

**API use.** POST/api/auth/signup

**Rule behavior.** Creates or reuses the customer identity, hashes the password with the project hash function, inserts the Users row, assigns the Customer role, and records UserSignUp.

**Data effects.** Reads: dbo.Roles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job is involved. Signup is synchronous and atomic.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_SignUp\_UPDATED.sql

### dbo.sp\_User\_Login

**Purpose.** Authenticates one supported login identifier and issues an opaque server-side session token.

**Authorization/scope.** Public endpoint protected by the backend login rate limiter. Effective roles are recalculated rather than trusting assigned roles alone.

**API use.** POST/api/auth/login

**Rule behavior.** Validates the password hash, confirms active user/customer state, determines effective Customer/Employee/Admin/HighAdmin roles, inserts Sessions with a fixed ExpiresAt, and returns identity plus role context.

**Data effects.** Reads: dbo.Customer, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job expires sessions. Expiration is checked on every protected request by sp\_User\_ValidateSession.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_Login.sql

### dbo.sp\_User\_ValidateSession

**Purpose.** Validates a bearer token and constructs the authoritative session context used as req.user.

**Authorization/scope.** Every protected request. It may additionally check a RequiredRole parameter.

**API use.** Allprotected/apiroutesthroughauthenticate()

**Rule behavior.** Finds the active session, rejects expired tokens, deactivates expired rows, recalculates roles and current branch, and returns UserID, CustomerID, EmployeeID, roles, and branch context.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles,

dbo.Sessions, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No scheduled cleanup is required for correctness because validation is request-time and authoritative.

**Source.** database/scripts/StoredProcedures/Session/Procedure\_User\_ValidateSession.sql

#### dbo.sp\_User\_Logout

**Purpose.** Revokes one session token.

**Authorization/scope.** Authenticated caller owning the presented bearer token.

**API use.** POST/api/auth/logout

**Rule behavior.** Marks Sessions.IsActive=0, sets LogoutTime, and records UserLogout.

**Data effects.** Reads: dbo.Sessions. Writes: dbo.AuditLog, dbo.Sessions.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job is involved.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_User\_Logout.sql

#### dbo.sp\_HighAdmin\_CreateInitialUser

**Purpose.** Creates the first HighAdmin identity exactly once.

**Authorization/scope.** One-time setup endpoint; procedure refuses creation when a HighAdmin already exists.

**API use.** POST/api/setup/initial-high-admin

**Rule behavior.** Creates or reuses an active customer, creates/reuses a user, assigns Customer and HighAdmin, and audits HighAdminCreated in one transaction.

**Data effects.** Reads: dbo.Customer, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job is involved.

**Source.** database/scripts/StoredProcedures/UserProcedures/Procedure\_HighAdmin\_CreateInitialUser.sql

#### dbo.sp\_Customer\_Create

**Purpose.** Creates a customer master record.

**Authorization/scope.** Employee, Admin, or HighAdmin through the customer route; SQL validates the acting user.

**API use.** POST/api/customers

**Rule behavior.** Validates names, 10-digit NationalID, age/birth date, unique NationalID/-phone/email, inserts Customer, and records CustomerCreated.

**Data effects.** Reads: dbo.Customer. Writes: dbo.AuditLog, dbo.Customer.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No direct Agent job. The new customer becomes branch-visible only after an account is opened in that branch.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Create.sql

#### dbo.sp\_Customer\_Search

**Purpose.** Searches customers with branch-aware visibility.

**Authorization/scope.** Employee/Admin see customers linked to their current branch through accounts; HighAdmin sees all.

**API use.** GET/api/customers

**Rule behavior.** Normalizes filters, resolves effective privilege and branch, filters by name/NationalID/phone/email/active status, and records the search.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job. Results can change after account creation, closure, or branch-related account changes.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Search.sql

#### dbo.sp\_Customer\_GetByID

**Purpose.** Returns one authorized customer record.

**Authorization/scope.** Customer can read self; staff can read customers linked to current branch; HighAdmin can read any.

**API use.** Usedbybackend/customeroperationsanddirectSQLworkflows

**Rule behavior.** Resolves caller identity, applies self/branch/all scope, returns one customer, and records CustomerViewed.

**Data effects.** Reads: dbo.Account, dbo.Customer, dbo.EMPB, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/CustomerProcedures/Procedure\_Customer\_Search.sql

#### dbo.sp\_Customer\_Update

**Purpose.** Updates customer contact information while preserving identity integrity.

**Authorization/scope.** Customer self or branch-scoped staff/HighAdmin according to SQL checks.

**API use.** PUT/api/customers/:customerID

**Rule behavior.** Locks the target, validates unique phone/email and access scope, updates allowed attributes, and records CustomerUpdated.

**Data effects.** Reads: `dbo.Account`, `dbo.Customer`, `dbo.EMPB`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.Customer`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/CustomerProcedures/Procedure_Customer_Update.sql`

#### `dbo.sp_Customer_Delete`

**Purpose.** Soft-deactivates a customer and disables associated application access.

**Authorization/scope.** Scoped Employee/Admin or HighAdmin; protected by pending-financial-operation rules.

**API use.** `DELETE/api/customers/:customerID`

**Rule behavior.** Checks scope and business dependencies, sets `Customer.IsActive=0`, deactivates linked `Users` where appropriate, and writes an audit record.

**Data effects.** Reads: `dbo.Account`, `dbo.Customer`, `dbo.EMPB`, `dbo.Sessions`, `dbo.Transactions`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.Customer`, `dbo.Users`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job. The customer trigger independently blocks deactivation when pending transactions exist.

**Source.** `database/scripts/StoredProcedures/CustomerProcedures/Procedure_Customer_Delete.sql`

#### `dbo.sp_AccountType_ListOptions`

**Purpose.** Returns safe account-product rows for dropdown controls.

**Authorization/scope.** Any authenticated user.

**API use.** `GET/api/accounts/options/account-types`

**Rule behavior.** Returns `AccountTypeID`, `TypeName`, description, minimum balance, interest rate, and monthly fee without allowing client-defined SQL identifiers.

**Data effects.** Reads: `dbo.AccountType`, `dbo.Customer`, `dbo.Users`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job; the returned `InterestRate` later affects the monthly-interest procedure.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountType_ListOptions.sql`

#### `dbo.sp_Account_Open`

**Purpose.** Opens an account for the authenticated customer.

**Authorization/scope.** Customer-owned identity; the branch and product are selected, but ownership is derived from `UserID`.

**API use.** `POST/api/accounts`

**Rule behavior.** Generates a global 12-digit zero-padded account number, validates branch/product and initial deposit, inserts the account at the product minimum balance, optionally initiates an initial deposit transaction, and audits the operation.

**Data effects.** Reads: `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending-transaction Agent may later finalize the optional initial deposit.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Open.sql`

#### `dbo.sp_Account_Search`

**Purpose.** Searches account portfolios with self, branch, or all-account scope.

**Authorization/scope.** Customer: own; Employee/Admin: current branch; HighAdmin: all.

**API use.** `GET/api/accountsandGET/api/accounts/mineviaaccess-scope.js`

**Rule behavior.** Applies rich filters, blocks customer cross-account access, calculates pending outgoing amount and available balance, orders results, and records `AccountSearch`.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Employee`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No direct Agent call. Pending-batch completion can change posted and available balances between searches.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Search.sql`

#### `dbo.sp_Account_GetInfo`

**Purpose.** Returns one account with ownership/branch/all authorization.

**Authorization/scope.** Same scope model as account search.

**API use.** `GET/api/accounts/:accountID`

**Rule behavior.** Resolves the caller and target, rejects unauthorized direct IDs, returns account/customer/branch/type details and available balance, and records `ACCOUNT_VIEW`.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Branch`, `dbo.Customer`, `dbo.Employee`, `dbo.Roles`, `dbo.TransactionType`, `dbo.Transactions`, `dbo.UserRoles`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_GetInfo.sql`

#### `dbo.sp_Transaction_GetAccountHistory`

**Purpose.** Returns newest-first transaction history for one authorized account.

**Authorization/scope.** Self, current branch, or all depending on effective role and requested scope.

**API use.** `GET/api/accounts/:accountID/history`

**Rule behavior.** Validates date range and account visibility, then returns debit and credit participation with transaction timestamps/status.

**Data effects.** Reads: `dbo.Account`, `dbo.Users`, `dbo.vw_AccountTransactionHistory`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The pending transaction Agent changes Pending rows to Completed/Failed, which is reflected in subsequent history reads.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountHistory.sql`

#### `dbo.sp_Account_GetAvailableBalance`

**Purpose.** Calculates spendable balance after pending outgoing holds.

**Authorization/scope.** Authorized account viewer.

**API use.** Used by account and transaction workflows

**Rule behavior.** Subtracts pending outgoing withdrawals/transfers from posted balance. This prevents double spending while transactions wait for finalization.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Users`, `dbo.vw_AccountPendingOutgoing`.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The pending transaction Agent releases the hold by completing or failing the transaction.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountHistory.sql`

#### `dbo.sp_Account_Freeze`

**Purpose.** Freezes an account while remembering whether it was Active or Dormant.

**Authorization/scope.** Employee/Admin within current branch; HighAdmin across all branches.

**API use.** `POST/api/accounts/:accountID/freeze`

**Rule behavior.** Locks the account, rejects Closed/already Frozen states, copies `AccountStatus` to `FrozenPreviousStatus`, sets `Frozen`, and audits `AccountFrozen`.

**Data effects.** Reads: `dbo.Account`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job directly changes Frozen accounts.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql`

#### `dbo.sp_Account_Unfreeze`

**Purpose.** Restores a frozen account to its previous Active or Dormant status.

**Authorization/scope.** Admin/branch manager within current branch or HighAdmin.

**API use.** `POST/api/accounts/:accountID/unfreeze`

**Rule behavior.** Locks the account, requires `Frozen` status, restores `FrozenPreviousStatus`,



clears the saved value, and audits AccountUnfrozen.

**Data effects.** Reads: `dbo.Account`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** No Agent job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql`

#### `dbo.sp_Account_Close`

**Purpose.** Closes an account safely.

**Authorization/scope.** Employee/Admin within current branch or HighAdmin; no financial transaction privilege is implied.

**API use.** `POST/api/accounts/:accountID/close`

**Rule behavior.** Locks the account, rejects Frozen/already Closed/pending-transaction states, sets balance to zero and status Closed with CloseDate, and audits AccountClosed.

**Data effects.** Reads: `dbo.Account`, `dbo.Transactions`, `dbo.Users`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** The account-ledger trigger records the balance delta automatically. The pending Agent must clear pending transactions before closure is allowed.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Close.sql`

#### `dbo.sp_Account_ChangeType`

**Purpose.** Changes the product attached to an account.

**Authorization/scope.** Employee/Admin within current branch or HighAdmin.

**API use.** `POST/api/accounts/:accountID/change-type`

**Rule behavior.** Requires Active or Dormant status, no pending transaction, a valid target AccountType, and enough available balance for the new minimum; then updates AccountTypeID and audits AccountTypeChanged.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.Transactions`, `dbo.vw_AccountPendingOutgoing`. Writes: `dbo.Account`, `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** Monthly interest behavior changes on the next run because the new AccountType.InterestRate is used.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ChangeType.sql`

#### `dbo.sp_Account_DormantSweep`

**Purpose.** Marks inactive eligible accounts Dormant.

**Authorization/scope.** Manual Admin/HighAdmin maintenance endpoint.

**API use.** `POST/api/accounts/maintenance/dormant-sweep`

**Rule behavior.** Finds Active accounts without qualifying completed customer activity for the configured month threshold, updates status, and audits each transition. Interest transactions do not count as customer activity.

**Data effects.** Reads: `dbo.Account`, `dbo.TransactionType`, `dbo.Transactions`. Writes: `dbo.AuditLog`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** There is no SQL Server Agent script for the dormant sweep in the current project; it is manual maintenance.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_DormantSweep.sql`

#### `dbo.sp_Account_ApplyMonthlyInterest`

**Purpose.** Creates monthly interest credits for eligible Active accounts.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** `POST/api/accounts/maintenance/apply-monthly-interest`

**Rule behavior.** Selects Active accounts whose `AccountType.InterestRate` is positive, calculates monthly interest, and inserts Interest transactions atomically.

**Data effects.** Reads: `dbo.Account`, `dbo.AccountType`, `dbo.TransactionType`. Writes: `dbo.AuditLog`, `dbo.Transactions`.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called by the Agent job Apply Monthly Account Interest on day 1 at 00:05. Resulting transactions may be finalized by the pending-transaction job.

**Source.** `database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ApplyMonthlyInterest.sql`

#### `dbo.sp_Transaction_Deposit`

**Purpose.** Initiates an owner-authorized deposit.

**Authorization/scope.** Only the account owner; staff roles do not grant transaction authority.

**API use.** `POST/api/transactions/deposit`

**Rule behavior.** Validates caller/customer ownership, positive amount, destination state, transaction type, computes `ReadyToCompleteAt`, inserts Pending transaction, and audits `DepositInitiated`.

**Data effects.** Reads: `dbo.Account`, `dbo.TransactionType`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.Transactions`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: `HOLDLOCK`, `UPDLOCK`.

**Automation relationship.** The Process Pending Bank Transactions Agent finalizes it when ready.

**Source.** `database/scripts/StoredProcedures/TransactionProcedures/Desposit_SP.sql`

#### `dbo.sp_Transaction-Withdrawal`

**Purpose.** Initiates an owner-authorized withdrawal.

**Authorization/scope.** Only the account owner.

**API use.** POST/api/transactions/withdraw

**Rule behavior.** Locks account and pending holds, requires Active source, positive amount, and enough available balance to preserve AccountType.MinBalance; inserts Pending withdrawal and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent finalizes it; until then the amount is reserved in available balance.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Withdraw\_SP.sql

#### dbo.sp\_Transaction\_Transfer

**Purpose.** Initiates an owner-authorized transfer.

**Authorization/scope.** Only the source-account owner; destination ownership is not required.

**API use.** POST/api/transactions/transfer

**Rule behavior.** Validates distinct accounts, source Active, destination not Closed/Frozen, source ownership, available funds/minimum balance, computes delay, inserts Pending transfer, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.TransactionType, dbo.Users, dbo.vw\_AccountPendingOutgoing. Writes: dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The pending-transaction Agent posts both account balance changes when ReadyToCompleteAt is reached.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Transfer\_SP.sql

#### dbo.sp\_Transaction\_Finalize

**Purpose.** Posts or fails one pending transaction.

**Authorization/scope.** Owner for manual endpoint, or trusted internal batch behavior.

**API use.** POST/api/transactions/:transactionID/finalize

**Rule behavior.** Locks the transaction and affected accounts, requires Pending and ready state, applies deposit/withdrawal/transfer/interest balance effects, marks Completed or Failed, writes CompletedAt, reactivates Dormant incoming accounts when applicable, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Customer, dbo.Transactions, dbo.Users. Writes: dbo.Account, dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** The batch procedure uses equivalent logic for Agent-driven processing. Account balance changes fire the branch-ledger trigger; transaction status changes fire the installment trigger.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/TransactionFinalizer.sql

**dbo.sp\_Transaction\_ProcessPendingBatch**

**Purpose.** Finalizes all ready pending transactions as one scheduled maintenance sweep.

**Authorization/scope.** Manual Admin/HighAdmin or SQL Server Agent.

**API use.** POST/api/transactions/maintenance/process-pending-batch

**Rule behavior.** Selects ready Pending rows with update locks, processes each type set-wise, marks Completed/Failed, updates accounts, and writes audit entries.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Transactions. Writes: dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** Called every minute by Process Pending Bank Transactions.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Procedure\_ProcessPendingBatch.sql

**dbo.sp\_Transaction\_Reverse**

**Purpose.** Cancels a pending transaction or reverses a completed one.

**Authorization/scope.** Only the transaction account owner under the current policy.

**API use.** POST/api/transactions/:transactionID/reverse

**Rule behavior.** Locks transaction/accounts, permits only Pending or Completed, reverses posted balance effects while protecting minimum balance, handles linked installment payment state, marks Cancelled, and audits.

**Data effects.** Reads: dbo.Account, dbo.AccountType, dbo.Customer, dbo.TransactionType, dbo.Transactions, dbo.Users. Writes: dbo.Account, dbo.AuditLog, dbo.Transactions.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job reverses transactions. Trigger effects occur when account balances or transaction statuses change.

**Source.** database/scripts/StoredProcedures/TransactionProcedures/Procedure\_ReverseTransaction.sql

**dbo.sp\_Loan\_Create**

**Purpose.** Creates a loan and complete installment schedule.

**Authorization/scope.** Employee/Admin in current branch; HighAdmin may choose any branch.

**API use.** POST/api/loans

**Rule behavior.** Validates active customer, branch, amount, rate, count, and dates; inserts Loan and evenly calculated Installment rows in one transaction; audits LoanCreated.

**Data effects.** Reads: dbo.Branch, dbo.Customer. Writes: dbo.AuditLog, dbo.Installment, dbo.Loan.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent creates loans. The overdue Agent later monitors the generated schedule.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_Create.sql

**dbo.sp\_Loan\_GetStatus**

**Purpose.** Returns loan summary and installment schedule.

**Authorization/scope.** Borrower sees own; staff visibility is prevalidated to current branch; HighAdmin sees all.

**API use.** GET/api/loans/:loanID/status

**Rule behavior.** Returns separate recordsets for loan and installments, including payment transaction readiness and dates. SQL independently prevents customer cross-loan access.

**Data effects.** Reads: dbo.Customer, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** The payment and overdue jobs change statuses later reflected here.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_GetStatus.sql

**dbo.sp\_Loan\_PayInstallment**

**Purpose.** Initiates payment of one installment using a borrower-owned account.

**Authorization/scope.** Borrower only; the payment account and loan customer must match the authenticated CustomerID.

**API use.** POST/api/loans/installments/:installmentID/pay

**Rule behavior.** Locks installment/loan/account, rejects Paid/Defaulted or duplicate in-flight payment, calls withdrawal logic, stores PaymentTransactionID, and audits the initiated payment.

**Data effects.** Reads: dbo.Account, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog, dbo.Installment.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** The pending transaction Agent finalizes the withdrawal. TR\_Transactions\_SyncInstallmentStatus then marks the installment Paid or releases a failed payment link.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_PayInstallment.sql

**dbo.sp\_Loan\_ProcessOverdueInstallments**

**Purpose.** Marks overdue installments Late or Defaulted and defaults loans where required.

**Authorization/scope.** Manual branch manager/Admin current branch; HighAdmin all or selected branch; Agent all branches.

**API use.** POST/api/loans/maintenance/process-overdue-installments

**Rule behavior.** Resolves manual scope when UserID is supplied, ignores installments with active in-flight payments, marks past-due rows Late and rows over 90 days Defaulted, updates affected loans, and audits the sweep.

**Data effects.** Reads: dbo.EMPB, dbo.Installment, dbo.Loan, dbo.Transactions, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** Called daily at 00:00 by Process Overdue Loan Installments. The Agent invokes it without UserID/BranchID, which is the trusted all-branch mode.

**Source.** database/scripts/StoredProcedures/LoanProcedures/Procedure\_Loan\_ProcessOverdueInstallments.sql

#### dbo.sp\_Branch\_ListOptions

**Purpose.** Returns non-sensitive branches for UI dropdowns.

**Authorization/scope.** Any authenticated user.

**API use.** GET/api/branches/options

**Rule behavior.** Returns BranchID, code, name, and city only.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.Users.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Branch\_ListOptions.sql

#### dbo.sp\_Branch\_GetInfo

**Purpose.** Returns branch operational details according to role.

**Authorization/scope.** Employee/Admin: current branch; HighAdmin: all or selected.

**API use.** GET/api/branchesandGET/api/branches/:branchID

**Rule behavior.** Resolves current assignment, prevents cross-branch staff access, returns branch data and manager context, and audits BranchInfoViewed.

**Data effects.** Reads: dbo.Account, dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** Balances may change indirectly when transaction or interest jobs update accounts and the ledger trigger updates Branch.Balance.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Branch\_GetInfo.sql

#### dbo.sp\_Employee\_Hire

**Purpose.** Creates an ordinary employee and current branch assignment.

**Authorization/scope.** Active Branch Manager/Vice Manager for own current branch. High-Admin generic employee hiring is intentionally disabled in the route.

**API use.** POST/api/employees

**Rule behavior.** Rejects manager-level titles, derives branch from the manager, inserts Employee and EMPB Working assignment, and audits the hire. No user login is created yet.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Hire.sql

**dbo.sp\_Employee\_CreateUserAccount**

**Purpose.** Creates an application login for an existing ordinary employee.

**Authorization/scope.** Scoped Admin or HighAdmin, with branch and manager-level restrictions.

**API use.** POST/api/employees/:employeeID/create-user-account

**Rule behavior.** Creates/reuses Customer, creates/reuses Users, links EmployeeID/CustomerID, assigns Customer and Employee roles, and audits EmployeeUserAccountCreated.

**Data effects.** Reads: dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_CreateUserAccount.sql

**dbo.sp\_Employee\_Search**

**Purpose.** Searches employee records with strict target scope.

**Authorization/scope.** Normal Employee: self; Admin: current branch; Vice Manager cannot inspect Branch Manager; HighAdmin: all.

**API use.** GET/api/employees

**Rule behavior.** Normalizes filters, optionally searches assignment history, applies current-branch rules, and audits the search.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Search.sql

**dbo.sp\_Employee\_GetInfo**

**Purpose.** Returns one employee and current assignment details.

**Authorization/scope.** Self/current branch/all depending on effective role, including Vice Manager protection.

**API use.** GET/api/employees/:employeeID

**Rule behavior.** Resolves target scope and returns employee information, current branch, and administrative flags; audits EmployeeInfoViewed.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_GetInfo.sql



**dbo.sp\_EMPBranchHistory\_Get**

**Purpose.** Returns temporal branch assignment history.

**Authorization/scope.** Self/current branch/all with manager hierarchy protection.

**API use.** GET/api/employees/:employeeID/branch-history

**Rule behavior.** Reads EMPB rows, optionally current-only, validates branch scope and target hierarchy, and audits EMPBHistoryViewed.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog.

**Transaction/concurrency.** Does not require an explicit multi-statement transaction in its current implementation.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EMPBranchHistory\_Get.sql

**dbo.sp\_Employee\_ChangeJobTitle**

**Purpose.** Changes an ordinary employee job title.

**Authorization/scope.** Scoped Admin/HighAdmin; manager-level titles are reserved for High-Admin governance procedures.

**API use.** POST/api/employees/:employeeID/change-job-title

**Rule behavior.** Locks target employee, validates branch and title restrictions, updates JobTitle, lets the admin-flag trigger synchronize CanAccessAdmin, and audits.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_ChangeJobTitle.sql

**dbo.sp\_Employee\_Suspend**

**Purpose.** Places an employee OnLeave.

**Authorization/scope.** Scoped Admin/HighAdmin; managers cannot suspend themselves and ordinary procedure cannot govern manager hierarchy improperly.

**API use.** POST/api/employees/:employeeID/suspend

**Rule behavior.** Locks target, validates current status/scope, sets EmpStatus=OnLeave, and audits. Session validation then removes effective Employee/Admin privileges while preserving Customer access.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job; role loss is enforced on the next request.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Suspend.sql



**dbo.sp\_Employee\_Fire**

**Purpose.** Terminates an employee and closes the current assignment.

**Authorization/scope.** Scoped Admin/HighAdmin with self/manager restrictions.

**API use.** POST/api/employees/:employeeID/fire

**Rule behavior.** Sets EmpStatus=Terminated, ends current EMPB row, and audits. The customer login may remain active but no longer receives effective staff roles.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_Employee\_Fire.sql

**dbo.sp\_EmployeeTransfer\_RequestByEmployee**

**Purpose.** Creates a two-stage transfer request for the authenticated employee.

**Authorization/scope.** Active Employee/Admin requesting for self.

**API use.** POST/api/employee-transfers/request-by-employee

**Rule behavior.** Derives employee/current branch from UserID, requires a different valid destination and no open request, inserts PendingCurrentManagerApproval, and audits.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job; both approvals are manual.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_RequestByEmployee.sql

**dbo.sp\_EmployeeTransfer\_RequestByManager**

**Purpose.** Creates a transfer request for an employee in the manager current branch.

**Authorization/scope.** Admin/HighAdmin, with current-branch rules for manager calls.

**API use.** POST/api/employee-transfers/request-by-manager

**Rule behavior.** Validates target employee, source/destination, open-request uniqueness, inserts the request, and audits.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_RequestByManager.sql

**dbo.sp\_EmployeeTransfer\_ApproveCurrentManager**

**Purpose.** Performs first-stage source-branch approval.

**Authorization/scope.** Current source Branch Manager/Vice Manager.

**API use.** POST/api/employee-transfers/:id/current-manager-decision

**Rule behavior.** Locks the request, requires PendingCurrentManagerApproval and source-branch authority, records decision/date/note, advances to destination approval or rejects, and audits.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_ApproveCurrentManager.sql

**dbo.sp\_EmployeeTransfer\_ApproveDestinationManager**

**Purpose.** Performs destination approval and executes the transfer.

**Authorization/scope.** Destination Branch Manager/Vice Manager.

**API use.** POST/api/employee-transfers/:id/destination-manager-decision

**Rule behavior.** Locks request and assignment, records decision, and on approval ends old EMPB, creates new Working EMPB, sets EffectiveDate/CompletedAt, and audits.

**Data effects.** Reads: dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job. EMPB overlap and one-manager triggers enforce temporal integrity.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_ApproveDestinationManager.sql

**dbo.sp\_HighAdmin\_HireManager**

**Purpose.** Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.

**Authorization/scope.** HighAdmin only; BranchID is mandatory and validated.

**API use.** POST/api/highadmin/managers

**Rule behavior.** Creates/reuses Customer, inserts Employee and EMPB for selected branch, creates/reuses Users, assigns Customer+Employee+Admin, validates manager title and branch uniqueness, and audits.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles, dbo.Users.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_HireManager.sql

#### dbo.sp\_HighAdmin\_PromoteToManager

**Purpose.** Promotes an active employee with login to manager level.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/promote

**Rule behavior.** Validates target and branch, updates title/admin capability, adjusts assignment when necessary, ensures Admin role, and audits EmployeePromotedToManager.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_PromoteToManager.sql

#### dbo.sp\_HighAdmin\_DowngradeManager

**Purpose.** Removes manager status and returns the employee to an ordinary title.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/downgrade

**Rule behavior.** Validates a non-manager replacement title, updates JobTitle/CanAccessAdmin, removes effective Admin behavior through triggers/role rules, and audits ManagerDowngraded.

**Data effects.** Reads: dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_DowngradeManager.sql

#### dbo.sp\_HighAdmin\_SuspendManager

**Purpose.** Places a manager OnLeave.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/suspend

**Rule behavior.** Validates manager target, updates status, and audits ManagerSuspended. Effective Admin/Employee roles disappear on session revalidation.

**Data effects.** Reads: dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_SuspendManager.sql

**dbo.sp\_HighAdmin\_FireManager**

**Purpose.** Terminates a manager and closes branch assignment.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/managers/:employeeID/fire

**Rule behavior.** Sets Terminated, ends current EMPB, and audits ManagerFired while preserving customer identity.

**Data effects.** Reads: dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee.

**Transaction/concurrency.** Uses an explicit SQL transaction.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_FireManager.sql

**dbo.sp\_HighAdmin\_ReplaceBranchManager**

**Purpose.** Atomically replaces the current Branch Manager.

**Authorization/scope.** HighAdmin only.

**API use.** POST/api/highadmin/branches/:branchID/replace-manager

**Rule behavior.** Locks branch/current assignment, downgrades or reassigns old manager, promotes/assigns replacement, enforces one active manager, and audits BranchManagerReplaced.

**Data effects.** Reads: dbo.Branch, dbo.Customer, dbo.EMPB, dbo.Employee, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EMPB, dbo.Employee, dbo.UserRoles.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_HighAdmin\_ReplaceBranchManager.sql

## Chapter 18

# EmployeeTransferRequest

EmployeeTransferRequest is a controlled two-stage workflow that preserves source/destination decisions and executes the actual EMPB change only after both approvals.

### 18.1 Attributes and structural constraints

#### Attributes of `dbo.EmployeeTransferRequest`

| Attribute                      | Type          | Constraint/default                     | Application meaning         |
|--------------------------------|---------------|----------------------------------------|-----------------------------|
| TransferRequestID              | INTIDENTITY   | PK, not null                           | Workflow identifier.        |
| EmployeeID                     | INT           | FK Employee                            | Employee being moved.       |
| FromBranchID                   | INT           | FK Branch; must differ from ToBranchID | Source.                     |
| ToBranchID                     | INT           | FK Branch                              | Destination.                |
| RequestedByUserID              | INT           | FK Users                               | Requester.                  |
| RequestType                    | NVARCHAR(30)  | EmployeeRequest or ManagerRequest      | Initiation mode.            |
| Reason                         | NVARCHAR(500) | Nullable                               | Reason.                     |
| CurrentManagerUserID           | INT           | Nullable FK Users                      | Source decision actor.      |
| CurrentManagerDecision         | NVARCHAR(20)  | NULL/Approved/Rejected                 | First decision.             |
| CurrentManagerDecisionDate     | DATETIME      | Nullable                               | Decision time.              |
| CurrentManagerNote             | NVARCHAR(500) | Nullable                               | Note.                       |
| DestinationManagerUserID       | INT           | Nullable FK Users                      | Destination decision actor. |
| DestinationManagerDecision     | NVARCHAR(20)  | NULL/Approved/Rejected                 | Second decision.            |
| DestinationManagerDecisionDate | DATETIME      | Nullable                               | Decision time.              |
| DestinationManagerNote         | NVARCHAR(500) | Nullable                               | Note.                       |
| TransferStatus                 | NVARCHAR(50)  | Workflow status set                    | Overall state.              |
| EffectiveDate                  | DATE          | Nullable                               | Actual transfer date.       |
| CreatedAt                      | DATETIME      | Default current time                   | Creation.                   |
| CompletedAt                    | DATETIME      | Nullable                               | Completion.                 |
| CancelledAt                    | DATETIME      | Nullable                               | Cancellation.               |

## 18.2 Entity rules

- FromBranchID and ToBranchID must differ.
- RequestType is EmployeeRequest or ManagerRequest.
- Only one open request per employee is permitted by a filtered unique index.
- Employee request derives the employee and source branch from authenticated UserID; manager request validates target is in the manager scope.
- First stage belongs to source Branch Manager/Vice Manager and requires PendingCurrentManagerApproval.
- Approval advances to PendingDestinationManagerApproval; rejection terminates as RejectedByCurrentManager.
- Second stage belongs to destination Branch Manager/Vice Manager; approval executes EMPB transfer atomically, rejection terminates as RejectedByDestinationManager.
- Completed sets EffectiveDate and CompletedAt; old assignment becomes Transferred and new assignment becomes Working.
- Stale IDs and repeated approvals are rejected because the procedure requires the expected current status.

## 18.3 TransferStatus workflow

| Status                            | Effect                                                                                  |
|-----------------------------------|-----------------------------------------------------------------------------------------|
| PendingCurrentManagerApproval     | Only source manager can decide.                                                         |
| PendingDestinationManagerApproval | Source approved; only destination manager can decide.                                   |
| RejectedByCurrentManager          | Terminal; no EMPB change.                                                               |
| RejectedByDestinationManager      | Terminal; no EMPB change.                                                               |
| Completed                         | Old EMPB becomes Transferred, new EMPB Working row inserted, dates and actors recorded. |
| Cancelled                         | Terminal cancellation without transfer.                                                 |

## 18.4 Procedures that enforce or expose these rules

### dbo.sp\_EmployeeTransfer\_RequestByEmployee

**Purpose.** Creates a two-stage transfer request for the authenticated employee.

**Authorization/scope.** Active Employee/Admin requesting for self.

**API use.** POST/api/employee-transfers/request-by-employee

**Rule behavior.** Derives employee/current branch from UserID, requires a different valid destination and no open request, inserts PendingCurrentManagerApproval, and audits.

**Data effects.** Reads: dbo.Branch, dbo.EMPB, dbo.Employee, dbo.EmployeeTransferRequest, dbo.Roles, dbo.UserRoles, dbo.Users. Writes: dbo.AuditLog, dbo.EmployeeTransferRequest.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job; both approvals are manual.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer

`_RequestByEmployee.sql`

#### `dbo.sp_EmployeeTransfer_RequestByManager`

**Purpose.** Creates a transfer request for an employee in the manager current branch.

**Authorization/scope.** Admin/HighAdmin, with current-branch rules for manager calls.

**API use.** POST/api/employee-transfers/request-by-manager

**Rule behavior.** Validates target employee, source/destination, open-request uniqueness, inserts the request, and audits.

**Data effects.** Reads: `dbo.Branch`, `dbo.EMPB`, `dbo.Employee`, `dbo.EmployeeTransferRequest`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EmployeeTransferRequest`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_RequestByManager.sql

#### `dbo.sp_EmployeeTransfer_ApproveCurrentManager`

**Purpose.** Performs first-stage source-branch approval.

**Authorization/scope.** Current source Branch Manager/Vice Manager.

**API use.** POST/api/employee-transfers/:id/current-manager-decision

**Rule behavior.** Locks the request, requires PendingCurrentManagerApproval and source-branch authority, records decision/date/note, advances to destination approval or rejects, and audits.

**Data effects.** Reads: `dbo.EMPB`, `dbo.Employee`, `dbo.EmployeeTransferRequest`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EmployeeTransferRequest`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer\_ApproveCurrentManager.sql

#### `dbo.sp_EmployeeTransfer_ApproveDestinationManager`

**Purpose.** Performs destination approval and executes the transfer.

**Authorization/scope.** Destination Branch Manager/Vice Manager.

**API use.** POST/api/employee-transfers/:id/destination-manager-decision

**Rule behavior.** Locks request and assignment, records decision, and on approval ends old EMPB, creates new Working EMPB, sets EffectiveDate/CompletedAt, and audits.

**Data effects.** Reads: `dbo.EMPB`, `dbo.Employee`, `dbo.EmployeeTransferRequest`, `dbo.Roles`, `dbo.UserRoles`, `dbo.Users`. Writes: `dbo.AuditLog`, `dbo.EMPB`, `dbo.EmployeeTransferRequest`.

**Transaction/concurrency.** Uses an explicit SQL transaction. Lock hints used: HOLDLOCK, UPDLOCK.

**Automation relationship.** No Agent job. EMPB overlap and one-manager triggers enforce

temporal integrity.

**Source.** database/scripts/StoredProcedures/Employees/Procedure\_EmployeeTransfer  
\_ApproveDestinationManager.sql

## 18.5 Triggers and scheduled automation

**Relevant triggers:** dbo.trg\_EMPB\_PreventDateOverlap, dbo.TR\_EMPB\_OneCurrentBranchManager.



## Chapter 19

# Cross-Entity Rule Chains

### 19.1 Signup, login, and session authorization

1. Signup creates/reuses Customer, creates Users, hashes the password, and assigns Customer role.
2. Login verifies the hash and inserts Sessions with fixed ExpiresAt.
3. Every protected request validates the token and recalculates effective roles from Users, Customer, Employee, UserRoles, and current EMPB.
4. Express attaches the result to req.user; route and SQL procedure checks then enforce ownership, branch, and hierarchy.

### 19.2 Ordinary employee hiring and login creation

1. Branch manager hires an ordinary employee; branch is derived from the manager session.
2. Employee and current EMPB row are created; no login exists yet.
3. Create-user-account creates/reuses Customer and Users, links EmployeeID, and assigns Customer and Employee.
4. Future session validation grants Employee only while EmpStatus=Active.

### 19.3 HighAdmin manager hiring

1. HighAdmin selects a mandatory BranchID and manager title.
2. Procedure validates branch and manager uniqueness.
3. Customer, Employee, EMPB, Users, and Customer+Employee+Admin roles are created/reused atomically.
4. Triggers verify valid links, role combinations, admin flag, and one active Branch Manager.

### 19.4 Account opening and product effects

1. Authenticated CustomerID becomes owner; selected BranchID and AccountTypeID are validated.
2. Sequence produces a 12-digit AccountNumber.
3. Product MinBalance determines required starting balance and future outgoing constraints.
4. Optional initial deposit becomes a Pending transaction and is later posted by the normal finalization pipeline.

### 19.5 Transaction posting and branch ledger

1. Owner initiates a typed Pending transaction.
2. Available balance immediately reserves pending outgoing amounts.
3. Finalizer or Agent rechecks current account states and minimum balances.
4. Account balances change atomically.

5. Account trigger updates Branch.Balance and inserts BranchLedger rows.
6. If transaction pays an installment, transaction trigger synchronizes Installment and possibly Loan.

## 19.6 Loan payment and default processing

1. Staff creates Loan and installment schedule for an authorized branch/customer.
2. Borrower selects a borrower-owned Active account and initiates payment.
3. Withdrawal transaction is linked through PaymentTransactionID.
4. Completion marks installment Paid; failure releases the link for retry.
5. Daily overdue job marks Late/Defaulted rows, ignoring protected in-flight payments, and defaults affected loans.

## 19.7 Employee transfer

1. Request validates employee, current source branch, distinct destination, and no open request.
2. Source manager approves/rejects.
3. Destination manager approves/rejects.
4. Approval closes old EMPB as Transferred and inserts new Working assignment in one transaction.
5. Session validation returns the new CurrentBranchID on the next request.

## Chapter 20

# Consolidated State and Type Matrices

### 20.1 Closed state sets versus lookup types

| Classification                                                               | Implementation                | Meaning                                                                                                   |
|------------------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------|
| Account products                                                             | AccountType rows              | Extensible data-driven types.                                                                             |
| Transaction categories                                                       | TransactionType rows          | Extensible lookup with trigger-enforced shape.                                                            |
| Application roles                                                            | Roles rows                    | Seeded catalogue with trigger-enforced combinations.                                                      |
| Account, transaction, loan, installment, employee, EMPB, and transfer states | CHECK-constrained text values | Closed lifecycle sets; adding a new state requires coordinated schema/procedure/backend/frontend changes. |

### 20.2 Account state operation matrix

| Operation                              | Active           | Dormant                             | Frozen              | Closed |
|----------------------------------------|------------------|-------------------------------------|---------------------|--------|
| Receive deposit/transfer               | Yes              | Yes, then re-activate on completion | No                  | No     |
| Withdraw/send transfer/pay installment | Yes, rules apply | No                                  | No                  | No     |
| Monthly interest                       | If rate > 0      | No                                  | No                  | No     |
| Freeze                                 | Yes              | Yes                                 | No                  | No     |
| Unfreeze                               | N/A              | N/A                                 | Admin or High-Admin | No     |
| Change type                            | Yes              | Yes                                 | No                  | No     |
| Close                                  | Yes              | Yes                                 | No                  | No     |

### 20.3 Employment and assignment matrix

| EmpStatus | WorkingStatus                                   | Current branch                    | Effective staff role                        |
|-----------|-------------------------------------------------|-----------------------------------|---------------------------------------------|
| Active    | Working                                         | Yes                               | Employee; Admin if manager conditions hold. |
| OnLeave   | Working                                         | Organizational assignment remains | Employee/Admin ineffective.                 |
| Active    | Transferred historical row plus new Working row | New branch                        | Based on new current row.                   |

| EmpStatus  | WorkingStatus | Current branch | Effective staff role        |
|------------|---------------|----------------|-----------------------------|
| Terminated | Ended         | No             | Employee/Admin ineffective. |

## 20.4 Loan and installment matrix

| Loan      | Installment       | Effect                                                              |
|-----------|-------------------|---------------------------------------------------------------------|
| Active    | Pending           | Payable by borrower; one in-flight transaction maximum.             |
| Active    | Late              | Payable by borrower; shown overdue.                                 |
| Defaulted | Defaulted present | Default reporting and normal defaulted installment payment blocked. |
| Paid      | All Paid          | No further installment payment.                                     |

## Chapter 21

# Complete Stored Procedure Inventory

The following table lists all stored procedures in the integrated project and summarizes the rule area they implement. Detailed behavior is described in the entity chapters.

| Procedure                           | Purpose and source                                                                                                                                                          | API/usage                                                      |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| dbo.sp_AccountType_ListOptions      | Returns safe account-product rows for drop-down controls.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountType_ListOptions.sql              | GET/api/accounts/options/account-types                         |
| dbo.sp_Account_ApplyMonthlyInterest | Creates monthly interest credits for eligible Active accounts.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ApplyMonthlyInterest.sql    | POST/api/accounts/maintenance/apply-monthly-interest           |
| dbo.sp_Account_ChangeType           | Changes the product attached to an account.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_ChangeType.sql                                 | POST/api/accounts/:accountID/change-type                       |
| dbo.sp_Account_Close                | Closes an account safely.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Close.sql                                                        | POST/api/accounts/:accountID/close                             |
| dbo.sp_Account_DormantSweep         | Marks inactive eligible accounts Dormant.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_DormantSweep.sql                                 | POST/api/accounts/maintenance/dormant-sweep                    |
| dbo.sp_Account_Freeze               | Freezes an account while remembering whether it was Active or Dormant.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql | POST/api/accounts/:accountID/freeze                            |
| dbo.sp_Account_GetAvailableBalance  | Calculates spendable balance after pending outgoing holds.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_AccountHistory.sql                      | Used by account and transaction workflows                      |
| dbo.sp_Account_GetInfo              | Returns one account with owner-ship/branch/all authorization.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_GetInfo.sql                  | GET/api/accounts/:accountID                                    |
| dbo.sp_Account_Open                 | Opens an account for the authenticated customer.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Open.sql                                  | POST/api/accounts                                              |
| dbo.sp_Account_Search               | Searches account portfolios with self, branch, or all-account scope.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Search.sql            | GET/api/accounts and GET/api/accounts/mine via access-scope.js |
| dbo.sp_Account_Unfreeze             | Restores a frozen account to its previous Active or Dormant status.<br>Source: database/scripts/StoredProcedures/AccountProcedures/Procedure_Account_Freeze_Unfreeze.sql    | POST/api/accounts/:accountID/unfreeze                          |

| Procedure                                         | Purpose and source                                                                                                                                                              | API/usage                                                    |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| dbo.sp_Branch_GetInfo                             | Returns branch operational details according to role.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Branch_GetInfo.sql                                        | GET/api/branchesandGET/api/branches/:branchID                |
| dbo.sp_Branch_ListOptions                         | Returns non-sensitive branches for UI dropdowns.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Branch_ListOptions.sql                                         | GET/api/branches/options                                     |
| dbo.sp_Customer_Create                            | Creates a customer master record.<br>Source: database/scripts/StoredProcedure/CustomerProcedures/Procedure_Customer_Create.sql                                                  | POST/api/customers                                           |
| dbo.sp_Customer_Delete                            | Soft-deactivates a customer and disables associated application access.<br>Source: database/scripts/StoredProcedure/CustomerProcedures/Procedure_Customer_Delete.sql            | DELETE/api/customers/:customerID                             |
| dbo.sp_Customer_GetByID                           | Returns one authorized customer record.<br>Source: database/scripts/StoredProcedure/CustomerProcedures/Procedure_Customer_Search.sql                                            | Usedbybackend/customeroperationsanddirectSQLworkflows        |
| dbo.sp_Customer_Search                            | Searches customers with branch-aware visibility.<br>Source: database/scripts/StoredProcedure/CustomerProcedures/Procedure_Customer_Search.sql                                   | GET/api/customers                                            |
| dbo.sp_Customer_Update                            | Updates customer contact information while preserving identity integrity.<br>Source: database/scripts/StoredProcedure/CustomerProcedures/Procedure_Customer_Update.sql          | PUT/api/customers/:customerID                                |
| dbo.sp_EMPBranchHistory_Get                       | Returns temporal branch assignment history.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_EMPBranchHistory_Get.sql                                            | GET/api/employees/:employeeID/branch-history                 |
| dbo.sp_EmployeeTransfer_ApproveCurrentManager     | Performs first-stage source-branch approval.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_EmployeeTransfer_ApproveCurrentManager.sql                         | POST/api/employee-transfers/:id/current-manager-decision     |
| dbo.sp_EmployeeTransfer_ApproveDestinationManager | Performs destination approval and executes the transfer.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_EmployeeTransfer_ApproveDestinationManager.sql         | POST/api/employee-transfers/:id/destination-manager-decision |
| dbo.sp_EmployeeTransfer_RequestByEmployee         | Creates a two-stage transfer request for the authenticated employee.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_EmployeeTransfer_RequestByEmployee.sql     | POST/api/employee-transfers/request-by-employee              |
| dbo.sp_EmployeeTransfer_RequestByManager          | Creates a transfer request for an employee in the manager current branch.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_EmployeeTransfer_RequestByManager.sql | POST/api/employee-transfers/request-by-manager               |
| dbo.sp_Employee_ChangeJobTitle                    | Changes an ordinary employee job title.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_ChangeJobTitle.sql                                             | POST/api/employees/:employeeID/change-job-title              |
| dbo.sp_Employee_CreateUserAccount                 | Creates an application login for an existing ordinary employee.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_CreateUserAccount.sql                  | POST/api/employees/:employeeID/create-user-account           |
| dbo.sp_Employee_Fire                              | Terminates an employee and closes the current assignment.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_Fire.sql                                     | POST/api/employees/:employeeID/fire                          |

| Procedure                             | Purpose and source                                                                                                                                                                      | API/usage                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| dbo.sp_Employee_GetInfo               | Returns one employee and current assignment details.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_GetInfo.sql                                               | GET/api/employees/:employeeID                         |
| dbo.sp_Employee_Hire                  | Creates an ordinary employee and current branch assignment.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_Hire.sql                                           | POST/api/employees                                    |
| dbo.sp_Employee_Search                | Searches employee records with strict target scope.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_Search.sql                                                 | GET/api/employees                                     |
| dbo.sp_Employee_Suspend               | Places an employee OnLeave.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_Employee_Suspend.sql                                                                        | POST/api/employees/:employeeID/suspend                |
| dbo.sp_HighAdmin_CreateInitialUser    | Creates the first HighAdmin identity exactly once.<br>Source: database/scripts/StoredProcedure/UserProcedures/Procedure_HighAdmin_CreateInitialUser.sql                                 | POST/api/setup/initial-high-admin                     |
| dbo.sp_HighAdmin_DowngradeManager     | Removes manager status and returns the employee to an ordinary title.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_DowngradeManager.sql                    | POST/api/highadmin/managers/:employeeID/downgrade     |
| dbo.sp_HighAdmin_FireManager          | Terminates a manager and closes branch assignment.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_FireManager.sql                                            | POST/api/highadmin/managers/:employeeID/fire          |
| dbo.sp_HighAdmin_HireManager          | Creates a Branch Manager or Vice Manager with branch assignment and login in one transaction.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_HireManager.sql | POST/api/highadmin/managers                           |
| dbo.sp_HighAdmin_PromoteToManager     | Promotes an active employee with login to manager level.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_PromoteToManager.sql                                 | POST/api/highadmin/managers/:employeeID/promote       |
| dbo.sp_HighAdmin_ReplaceBranchManager | Atomically replaces the current Branch Manager.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_ReplaceBranchManager.sql                                      | POST/api/highadmin/branches/:branchID/replace-manager |
| dbo.sp_HighAdmin_SuspendManager       | Places a manager OnLeave.<br>Source: database/scripts/StoredProcedure/Employees/Procedure_HighAdmin_SuspendManager.sql                                                                  | POST/api/highadmin/managers/:employeeID/suspend       |
| dbo.sp_Loan_Create                    | Creates a loan and complete installment schedule.<br>Source: database/scripts/StoredProcedure/LoanProcedures/Procedure_Loan_Create.sql                                                  | POST/api/loans                                        |
| dbo.sp_Loan_GetStatus                 | Returns loan summary and installment schedule.<br>Source: database/scripts/StoredProcedure/LoanProcedures/Procedure_Loan_GetStatus.sql                                                  | GET/api/loans/:loanID/status                          |
| dbo.sp_Loan_PayInstallment            | Initiates payment of one installment using a borrower-owned account.<br>Source: database/scripts/StoredProcedure/LoanProcedures/Procedure_Loan_PayInstallment.sql                       | POST/api/loans/installments/:installmentID/pay        |

| Procedure                              | Purpose and source                                                                                                                                                                       | API/usage                                               |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| dbo.sp_Loan_ProcessOverdueInstallments | Marks overdue installments Late or Defaulted and defaults loans where required.<br>Source: database/scripts/StoredProcedure/LoanProcedures/Procedure_Loan_ProcessOverdueInstallments.sql | POST/api/loans/maintenance/process-overdue-installments |
| dbo.sp_Transaction_Deposit             | Initiates an owner-authorized deposit.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/Desposit_SP.sql                                                                 | POST/api/transactions/deposit                           |
| dbo.sp_Transaction_Finalize            | Posts or fails one pending transaction.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/TransactionFinalizer.sql                                                       | POST/api/transactions/:transactionID/finalize           |
| dbo.sp_Transaction_GetAccountHistory   | Returns newest-first transaction history for one authorized account.<br>Source: database/scripts/StoredProcedure/AccountProcedures/Procedure_AccountHistory.sql                          | GET/api/accounts/:accountID/history                     |
| dbo.sp_Transaction_ProcessPendingBatch | Finalizes all ready pending transactions as one scheduled maintenance sweep.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/Procedure_ProcessPendingBatch.sql         | POST/api/transactions/maintenance/process-pending-batch |
| dbo.sp_Transaction_Reverse             | Cancels a pending transaction or reverses a completed one.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/Procedure_ReverseTransaction.sql                            | POST/api/transactions/:transactionID/reverse            |
| dbo.sp_Transaction_Transfer            | Initiates an owner-authorized transfer.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/Transfer_SP.sql                                                                | POST/api/transactions/transfer                          |
| dbo.sp_Transaction_Withdrawal          | Initiates an owner-authorized withdrawal.<br>Source: database/scripts/StoredProcedure/TransactionProcedures/Withdraw_SP.sql                                                              | POST/api/transactions/withdraw                          |
| dbo.sp_User_Login                      | Authenticates one supported login identifier and issues an opaque server-side session token.<br>Source: database/scripts/StoredProcedure/UserProcedures/Procedure_User_Login.sql         | POST/api/auth/login                                     |
| dbo.sp_User_Logout                     | Revokes one session token.<br>Source: database/scripts/StoredProcedure/UserProcedures/Procedure_User_Logout.sql                                                                          | POST/api/auth/logout                                    |
| dbo.sp_User_SignUp                     | Creates a self-service customer login and guarantees the Customer role.<br>Source: database/scripts/StoredProcedure/UserProcedures/Procedure_User_SignUp_UPDATED.sql                     | POST/api/auth/signup                                    |
| dbo.sp_User_ValidateSession            | Validates a bearer token and constructs the authoritative session context used as req.user.<br>Source: database/scripts/StoredProcedure/Session/Procedure_User_ValidateSession.sql       | Allprotected/apiroutes throughauthenticate()            |



## Chapter 22

# Trigger, Function, View, and Agent Inventory

### 22.1 Triggers

| Object                                   | Rule responsibility                                                              |
|------------------------------------------|----------------------------------------------------------------------------------|
| TR_Account_PreventCloseWithPending       | Blocks transition to Closed while a Pending transaction references the account.  |
| TR_Account_SyncBranchLedger              | Synchronizes Branch.Balance and BranchLedger from Account.Balance changes.       |
| TR_Customer_PreventDeactivateWithPending | Blocks deactivation while customer accounts participate in Pending transactions. |
| TR_Transactions_SyncInstallmentStatus    | Synchronizes installment/loan states from payment transaction status.            |
| TR_Transactions_EnforceTypePattern       | Enforces debit/credit account-reference pattern for each TransactionType.        |
| trg_EMPB_PreventDateOverlap              | Prevents overlapping branch-assignment periods for one employee.                 |
| TR_Employee_SetAdminFlag                 | Synchronizes CanAccessAdmin and removes invalid Admin roles.                     |
| TR_Users_EnforceUserLinks                | Enforces required Customer link and HighAdmin/Employee link rules.               |
| TR_UserRoles_EnforceAccessModel          | Enforces valid role combinations and manager-only Admin.                         |
| TR_EMPB_OneCurrentBranchManager          | Enforces one active Branch Manager when assignments change.                      |
| TR_Employee_OneCurrentBranchManager      | Enforces one active Branch Manager when employee title/status changes.           |

### 22.2 Functions and helper view

| Object                       | Responsibility                                                                         |
|------------------------------|----------------------------------------------------------------------------------------|
| fn_HashPassword              | Produces the 64-character SHA-256 password hash used by Users.                         |
| fn_UserHasEffectiveRole      | Calculates current Customer/Employee/Admin or HighAdmin effectiveness from live state. |
| fn_GetCompletionDelaySeconds | Maps amount to 0/60/180/300-second readiness delay.                                    |
| vw_AccountPendingOutgoing    | Aggregates Pending outgoing holds per account for available-balance calculations.      |

## 22.3 SQL Server Agent jobs

| Job                               | Schedule     | Effect                                                                          |
|-----------------------------------|--------------|---------------------------------------------------------------------------------|
| Process Pending Bank Transactions | Every minute | Calls sp_Transaction_ProcessPendingBatch to post/fail ready transactions.       |
| Process Overdue Loan Installments | Daily 00:00  | Calls sp_Loan_ProcessOverdueInstallments in trusted all-branch mode.            |
| Apply Monthly Account Interest    | Day 1, 00:05 | Calls sp_Account_ApplyMonthlyInterest to create eligible interest transactions. |

## Chapter 23

## Conclusion

- Types stored in lookup tables are extensible data; lifecycle states stored in CHECK constraints are closed workflow vocabularies.
- A state value is never merely visual. It changes authorization, available actions, automation eligibility, reports, and downstream triggers.
- Ownership and branch visibility are separate: staff can view/manage branch records, but financial transactions and installment payment remain owner/borrower only.
- Current privileges come from live database state, not from data embedded in the session token.
- Scheduled work reuses stored procedures rather than duplicating rules in job scripts.
- Cross-entity consistency is protected with transactions, locks, constraints, filtered unique indexes, and triggers.
- The implementation is direct SQL Server access through mssql, not an ORM abstraction.

## Source Basis

This document was prepared from the integrated project under `database/scripts`, `database/security`, and `src`. The schema, 52 stored procedures, access-model triggers, financial triggers, report routes, middleware, and SQL Server Agent scripts were checked against the current project source that includes patches 05 through 10.